HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CNTT-1

IOT và Ứng dụng



BÁO CÁO CUỐI MÔN

Đề tài:

GasGuard – Hệ thống IoT giám sát, cảnh báo rò rỉ khí gas, quá nhiệt và điều khiển quạt tản từ xa

Giảng viên hướng dẫn	**Kim Ngọc Bách**
Họ và tên sinh viên	**Mã sinh viên**
Đỗ Bình Minh	**B23DCCN542**
Nguyễn Đại Nghĩa	**B23DCCN598**
Lớp	**D23CQCN10-B**
Nhóm lớp	**09**

MỤC LỤC

THÔNG TIN NHÓM VÀ PHÂN CÔNG CÔNG VIỆC 3
TÓM TẮT ĐỒ ÁN 4
CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN 5
1.1. Bối cảnh và vấn đề cần giải quyết 5
1.2. Đối tượng sử dụng và các bên liên quan 5
1.3. Mục tiêu của hệ thống 5
1.4. Phạm vi triển khai 6
1.5. Vì sao cần giải pháp IoT 6
CHƯƠNG 2. KHẢO SÁT YÊU CẦU 6
2.1. Các kịch bản sử dụng chính 6
2.2. Sơ đồ use case 7
2.3. Yêu cầu chức năng 7
2.4. Yêu cầu phi chức năng 8
2.5. Các ràng buộc 8
2.6. Tiêu chí đánh giá thành công 9
CHƯƠNG 3. PHÂN TÍCH VÀ LỰA CHỌN GIẢI PHÁP 9
3.1. Sơ đồ bối cảnh hệ thống 9
3.2. Lựa chọn vi điều khiển và cảm biến 10
3.3. Lựa chọn giao thức 10
3.4. Lựa chọn broker, backend, cơ sở dữ liệu và giao diện 11
3.5. Vì sao không dùng nền tảng IoT có sẵn 11
CHƯƠNG 4. KIẾN TRÚC HỆ THỐNG 11
4.1. Kiến trúc tổng thể 12
4.2. Kiến trúc phân lớp 12
4.3. Luồng dữ liệu đầu-cuối 13
CHƯƠNG 5. THIẾT KẾ PHẦN CỨNG 13
5.1. Danh sách linh kiện 13
5.2. Sơ đồ khối phần cứng 14
5.3. Sơ đồ nguyên lý và bảng chân 14
5.5. Tính toán thiết kế 15
5.6. Thiết kế firmware 17
5.7. Hình ảnh nguyên mẫu 17
5.8. An toàn điện 19
CHƯƠNG 6. THIẾT KẾ GIAO THỨC VÀ LUỒNG DỮ LIỆU 19
6.1. Cấu trúc topic MQTT 19

6.2. Định dạng bản tin 20
6.3. QoS, thử lại, quá hạn và chống trùng 20
6.4. Phát hiện online/offline 21
6.5. Kiểm tra hợp lệ dữ liệu đầu vào 21
6.6. Sequence diagram các quy trình chính 21
CHƯƠNG 7. THIẾT KẾ CƠ SỞ DỮ LIỆU, API VÀ BACKEND 25
7.1. Mô hình dữ liệu (ERD) 25
7.2. Kiến trúc backend 26
7.3. Mô tả API 26
7.4. Rule engine cảnh báo 27
CHƯƠNG 8. QUẢN LÝ VÒNG ĐỜI THIẾT BỊ 27
8.1. Mô hình trạng thái 27
8.2. Quy trình đăng ký, cấp xác thực và kích hoạt 28
8.3. Quy trình tạm khóa và ngừng sử dụng 29
8.4. Đối chiếu yêu cầu quản lý vòng đời 32
CHƯƠNG 9. QUẢN LÝ NGƯỜI DÙNG VÀ PHÂN QUYỀN 32
9.1. Quản lý tài khoản 32
9.2. Ma trận vai trò và quyền 32
9.3. Kiểm tra quyền ở backend 33
CHƯƠNG 10. THIẾT KẾ BẢO MẬT VÀ QUYỀN RIÊNG TƯ 33
10.1. Mô hình mối đe dọa 33
10.2. Đáp ứng các yêu cầu bảo mật của đề bài 35
10.3. Quyền riêng tư 35
CHƯƠNG 11. TRIỂN KHAI HỆ THỐNG 36
11.1. Sơ đồ triển khai 36
11.2. Các bước triển khai 37
11.3. Giao diện người dùng 37
11.4. Quản lý mã nguồn 39
CHƯƠNG 12. KIỂM THỬ VÀ ĐÁNH GIÁ 39
12.1. Kế hoạch và môi trường kiểm thử 39
12.2. Kiểm thử chức năng 39
12.3. Kiểm thử bảo mật 40
12.4. Kiểm thử phi chức năng 40
12.5. Kiểm thử với thiết bị thật (ESP32) 41
12.6. Đánh giá chung 43
CHƯƠNG 13. CHỨC NĂNG NÂNG CAO 43

CHƯƠNG 14. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN 43
CHƯƠNG 15. KẾT LUẬN 44
TÀI LIỆU THAM KHẢO 44
PHỤ LỤC A. DANH SÁCH ĐẦY ĐỦ API 46
PHỤ LỤC B. DỮ LIỆU ĐO CHI TIẾT CÁC CHỈ TIÊU PHI CHỨC NĂNG 47
P1 – Độ trễ từ cảm biến đến dashboard 47
P2 – Độ trễ lệnh điều khiển 47
P3, P5 – Mất bản tin và bản tin trùng 48
P4 – Mất và khôi phục mạng 48
P6, P7 – Tài nguyên và độ ổn định 48
P8 – Số lượng thiết bị đồng thời 49
PHỤ LỤC C. LẮP RÁP TRÊN BREADBOARD VÀ SƠ ĐỒ HOẠT ĐỘNG FIRMWARE 49
PHỤ LỤC D. MÁY TRẠNG THÁI KẾT NỐI, LỆNH VÀ CẢNH BÁO 51
PHỤ LỤC E. THƯ VIỆN, TÀI NGUYÊN BÊN THỨ BA VÀ CÔNG CỤ AI 52

THÔNG TIN NHÓM VÀ PHÂN CÔNG CÔNG VIỆC

Bảng 1. *Thông tin thành viên*

TT	Họ và tên	MSSV	Vai trò chính
1	Đỗ Bình Minh	B23DCCN542	Phân tích bài toán và yêu cầu, phần cứng (chọn linh kiện, lắp mạch, sơ đồ), firmware ESP32, giao diện web React, kiểm thử với ESP32 thật (H1–H10)
2	Nguyễn Đại Nghĩa	B23DCCN598	Backend Spring Boot, cơ sở dữ liệu, API; broker Mosquitto, TLS, triển khai Docker Compose; kiểm thử tự động, thiết bị mô phỏng

Nhóm có 2 thành viên nên hệ thống triển khai **một node thiết bị vật lý** (ESP32 + MQ-2 + DS18B20 + relay/quạt + còi + LED). Khuyến nghị "từ hai node trở lên" chỉ áp dụng cho nhóm 4 thành viên. Khả năng mở rộng nhiều node được đánh giá bổ sung bằng thiết bị mô phỏng chạy đúng giao thức (tối đa 100 thiết bị đồng thời, mục 12.4).

Bảng 2. *Bảng phân công công việc*

Hạng mục	Đỗ Bình Minh	Nguyễn Đại Nghĩa
Phân tích bài toán, yêu cầu	Chính	Phối hợp
Phần cứng: chọn linh kiện, lắp mạch, sơ đồ (hardware/)	Chính	Phối hợp
Firmware ESP32 (firmware/gas-node)	Chính	Rà soát giao thức
Broker Mosquitto, TLS, Docker Compose (deployment/)	Phối hợp	Chính
Backend Spring Boot, CSDL, API (backend/)	Phối hợp	Chính
Giao diện web React (frontend/)	Chính	Phối hợp
Kiểm thử tự động, thiết bị mô phỏng (tests/)	Phối hợp	Chính
Kiểm thử với ESP32 thật (H1–H10)	Chính	Phối hợp
Báo cáo, slide, video demo (docs/)	Phối hợp	Phối hợp

TÓM TẮT ĐỒ ÁN

Rò rỉ khí gas (LPG) trong bếp gia đình, quán ăn và phòng trọ là nguyên nhân thường gặp của cháy nổ và ngộ độc. Đầu báo gas thông thường chỉ kêu tại chỗ: khi không có người, không ai biết; không có lịch sử để kiểm tra; không điều khiển được quạt hút từ xa; người quản lý nhiều điểm không theo dõi tập trung được. Đồ án xây dựng **GasGuard**, một nguyên mẫu IoT hoàn chỉnh giải quyết các hạn chế này.

Thiết bị là một node **ESP32-WROOM-32** đo nồng độ khí gas bằng **MQ-2** (đã hiệu chuẩn R0, quy đổi ppm theo đường đặc tuyến LPG) và nhiệt độ bằng **DS18B20**, điều khiển **quạt hút qua relay**, **còi** qua transistor và hai LED trạng thái. Firmware chạy luật báo động tại chỗ kể cả khi mất mạng, đệm 120 bản đo và gửi bù khi kết nối lại. Thiết bị giao tiếp với broker **Mosquitto** qua **MQTT trên TLS** (cổng 8883), mỗi thiết bị có tài khoản và ACL riêng do backend cấp động bằng Dynamic Security. Backend **Spring Boot 3.5 (Java 21)** tiếp nhận và kiểm tra telemetry, lưu vào **PostgreSQL 16**, chạy rule engine cảnh báo, watchdog offline, quản lý lệnh có xác nhận (ACK), cấu hình từ xa có phiên bản, vòng đời thiết bị 6 trạng thái, người dùng 3 vai trò với JWT và audit log. Giao diện **React** cập nhật thời gian thực qua WebSocket (STOMP) và gửi email cảnh báo qua Gmail. Toàn bộ hệ thống chạy bằng **Docker Compose**.

Kết quả kiểm thử tự động: **18/18 ca chức năng và bảo mật đạt** (F1–F9, S1–S9). Độ trễ cảm biến → dashboard P95 **24,5 ms**; độ trễ lệnh khứ hồi P95 **151,9 ms**; mất bản tin QoS1 **0 %**; phát hiện thiết bị mất kết nối đột ngột sau **1,0 s** nhờ LWT; gửi bù đủ 20/20 bản đo sau 20 s mất mạng; chạy liên tục 10 phút không mất bản tin. Ở mức 100 thiết bị mô phỏng (100 bản tin/s), hệ thống mất 8,22 % bản tin, cho thấy giới hạn

hiện tại nằm ở việc ghi CSDL từng giao dịch; nhóm phân tích nguyên nhân và đề xuất hướng khắc phục.

Từ khóa: IoT, ESP32, MQ-2, MQTT, TLS, Spring Boot, PostgreSQL, quản lý vòng đời thiết bị, cảnh báo rò rỉ khí gas.

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1. Bối cảnh và vấn đề cần giải quyết

Bếp gas LPG được dùng phổ biến trong hộ gia đình, phòng trọ, quán ăn và bếp ăn tập thể. Khi van, dây dẫn hoặc bếp bị hở, khí LPG nặng hơn không khí tích tụ sát sàn; chỉ cần một tia lửa (công tắc điện, bật bếp) là có thể gây cháy nổ. Các đầu báo gas độc lập trên thị trường giải quyết được việc phát hiện tại chỗ nhưng còn bốn hạn chế:

- **Chỉ báo động tại chỗ.** Khi chủ nhà đi vắng hoặc ngủ say, tiếng còi không đến được người có thể xử lý.
- **Không có lịch sử.** Không biết nồng độ gas đã tăng từ lúc nào, có tái diễn hay không, để kiểm tra rò rỉ nhỏ kéo dài.
- **Không tác động từ xa.** Không bật được quạt hút để giảm nồng độ trong lúc chờ người đến.
- **Không quản lý tập trung.** Chủ nhà trọ hoặc quản lý bếp có nhiều điểm đo không có một nơi theo dõi chung, không phân quyền ai được xem, ai được điều khiển.

1.2. Đối tượng sử dụng và các bên liên quan

Bảng 1.1. *Đối tượng sử dụng và nhu cầu*

Vai trò hệ thống	Ai trong thực tế	Nhu cầu chính
Quản trị viên (ADMIN)	Người vận hành hệ thống, đơn vị cung cấp dịch vụ	Quản lý tài khoản, toàn bộ thiết bị, luật cảnh báo chung, xem audit log và tình trạng hệ thống
Chủ thiết bị (OWNER)	Chủ hộ, chủ nhà trọ, quản lý bếp	Đăng ký, cấu hình, điều khiển quạt/còi, xử lý cảnh báo, chia sẻ quyền xem cho người khác
Người xem (VIEWER)	Người thân, người thuê phòng, nhân viên bếp	Xem dữ liệu, trạng thái và cảnh báo của thiết bị được chia sẻ; nhận email cảnh báo
Bên liên quan khác	Đội PCCC cơ sở, chủ cho thuê	Hưởng lợi gián tiếp từ việc phát hiện sớm và lịch sử sự cố

1.3. Mục tiêu của hệ thống

1. Phát hiện nồng độ khí gas và nhiệt độ bất thường trong vòng vài giây, báo động tại chỗ (còi, LED, tự bật quạt) ngay cả khi mất mạng.
2. Đưa dữ liệu về máy chủ theo thời gian thực, cảnh báo người dùng ở xa qua giao diện web và email.
3. Cho phép người có quyền điều khiển quạt, còi và thay đổi cấu hình thiết bị từ xa, có xác nhận kết quả.

4. Quản lý người dùng, phân quyền và toàn bộ vòng đời thiết bị từ đăng ký đến ngừng sử dụng, đảm bảo an toàn thông tin.
5. Lưu lịch sử đo, cảnh báo, lệnh và thao tác để tra cứu, thống kê và kiểm tra lại.

1.4. Phạm vi triển khai

Trong phạm vi: một hoặc nhiều node ESP32 đặt tại bếp, kết nối WiFi 2,4 GHz tới máy chủ chạy Docker trong cùng mạng LAN (nhà riêng, phòng trọ, hoặc hotspot khi demo). Hệ thống phục vụ quy mô hộ gia đình đến một khu nhà trọ (hàng chục thiết bị).

Ngoài phạm vi: thay thế thiết bị báo cháy đạt chuẩn PCCC; tự động khóa van gas (cần van điện từ chuyên dụng và chứng nhận an toàn); triển khai ra Internet công cộng với tên miền và chứng chỉ công cộng; cập nhật firmware từ xa (OTA) và chức năng AI.

1.5. Vì sao cần giải pháp IoT

Một ứng dụng web hoặc phần mềm quản lý thông thường không giải quyết được bài toán vì ba lý do. Thứ nhất, dữ liệu cần giám sát (nồng độ gas, nhiệt độ) **chỉ tồn tại tại hiện trường** và thay đổi theo từng giây, nên bắt buộc phải có thiết bị đo đặt tại bếp và gửi dữ liệu liên tục qua mạng. Thứ hai, phản ứng phải **tác động vào môi trường vật lý** (bật quạt hút, kêu còi) và phải xảy ra tại chỗ trong vài giây, kể cả khi mạng hoặc máy chủ gặp sự cố, nên một phần xử lý phải chạy ngay trên thiết bị (edge). Thứ ba, người chịu trách nhiệm thường **không ở hiện trường**, nên cần kết nối hai chiều: dữ liệu và cảnh báo đi lên người dùng ở xa, lệnh điều khiển và cấu hình đi xuống thiết bị, kèm cơ chế xác thực từng thiết bị và phân quyền người dùng. Ba yếu tố cảm biến – kết nối – tác động này chính là đặc trưng của một hệ thống IoT [2].

CHƯƠNG 2. KHẢO SÁT YÊU CẦU

2.1. Các kịch bản sử dụng chính

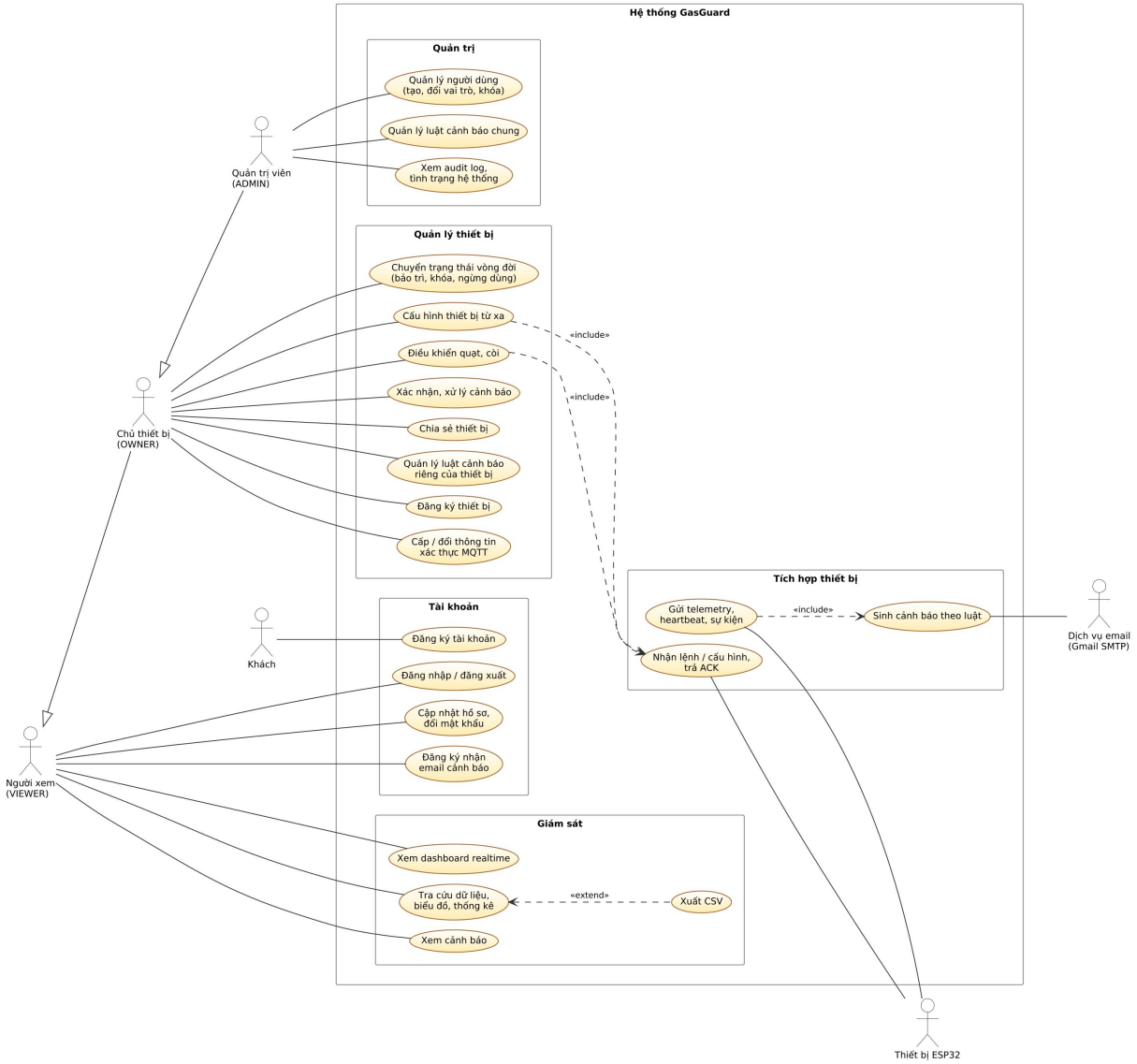
Bảng 2.1. *Kịch bản sử dụng chính*

Mã	Kịch bản	Diễn biến
KB1	Rò rỉ gas khi có người	MQ-2 đo vượt ngưỡng 400 ppm → thiết bị bật LED đỏ, còi, tự bật quạt trong chu kỳ đo; dashboard hiển thị cảnh báo WARNING; người dùng xác nhận và xử lý cảnh báo sau khi khóa van.
KB2	Rò rỉ gas khi vắng nhà	Như KB1, đồng thời backend gửi email tới chủ thiết bị và người được chia sẻ; chủ nhà mở web từ xa, xem biểu đồ, giữ quạt BẬT bằng lệnh FAN_SET.
KB3	Thiết bị mất điện / mất WiFi	Broker phát LWT, backend chuyển thiết bị sang OFFLINE và mở cảnh báo OFFLINE; khi có mạng lại, thiết bị gửi bù dữ liệu đo trong thời gian mất kết nối.
KB4	Lắp đặt thiết bị mới	Chủ thiết bị đăng ký mã thiết bị, cấp thông tin xác thực MQTT, nạp vào firmware; thiết bị tự kích hoạt khi gửi bản tin đầu tiên.

Mã	Kịch bản	Diễn biến
KB5	Thu hồi thiết bị	Thiết bị bị mất hoặc chuyển nhượng: chủ thiết bị tạm khóa hoặc ngừng sử dụng; broker ngắt kết nối ngay, mật khẩu cũ hết hiệu lực.
KB6	Chia sẻ cho người thân	Chủ thiết bị chia sẻ quyền xem; người thân nhận email cảnh báo, xem dữ liệu nhưng không điều khiển được.

2.2. Sơ đồ use case

Hình 2.1 trình bày các tác nhân và trường hợp sử dụng theo ký pháp UML [3]. Ba vai trò người dùng có quan hệ kế thừa: OWNER có mọi khả năng của VIEWER cộng thêm quyền quản lý thiết bị của mình; ADMIN có mọi khả năng của OWNER trên toàn bộ thiết bị cộng thêm quyền quản trị. Thiết bị ESP32 và dịch vụ email là tác nhân hệ thống.



Hình 2.1. *Sơ đồ use case của hệ thống GasGuard*

2.3. Yêu cầu chức năng

Bảng 2.2. *Yêu cầu chức năng*

Mã	Nhóm	Yêu cầu
FR1	Thiết bị	Đo gas (ppm) và nhiệt độ (°C) theo chu kỳ cấu hình được; gắn mã định danh, seq, timestamp UTC; gửi heartbeat 10 s; nhận lệnh, cấu hình và trả ACK; điều khiển quạt, còi; tự kết nối lại; đệm dữ liệu khi mất mạng.
FR2	Người dùng	Đăng ký, đăng nhập/đăng xuất, đổi mật khẩu và thông tin; admin tạo tài khoản, đổi vai trò, khóa/mở khóa; 3 vai trò.
FR3	Vòng đời thiết bị	Đăng ký, cấp thông tin xác thực, kích hoạt, bảo trì, tạm khóa, mở khóa, ngừng sử dụng, đổi mật khẩu thiết bị; gán chủ sở hữu, vị trí, khu vực; theo dõi online/offline, lỗi, phiên bản phần cứng/firmware.
FR4	Dữ liệu	Kiểm tra hợp lệ, loại trùng, lưu theo thời gian; tra cứu theo thiết bị và khoảng thời gian; biểu đồ, thống kê min/max/trung bình/P95; xuất CSV.
FR5	Cảnh báo	Luật ngưỡng chung và theo thiết bị; cảnh báo OFFLINE, lỗi cảm biến; lưu lịch sử; xác nhận, xử lý; gửi email.
FR6	Điều khiển	Gửi lệnh từ giao diện; kiểm tra quyền ở backend; lưu lịch sử lệnh, kết quả, độ trễ; quá hạn khi không có ACK.
FR7	Giao diện	Dashboard realtime; danh sách và chi tiết thiết bị; dữ liệu, biểu đồ; cảnh báo; cấu hình, điều khiển; người dùng; audit log.

2.4. Yêu cầu phi chức năng

Bảng 2.3. *Yêu cầu phi chức năng và chỉ tiêu định lượng*

Mã	Thuộc tính	Chỉ tiêu	Cách kiểm chứng
NFR1	Độ trễ cảm biến → dashboard	P95 ≤ 1 000 ms	P1
NFR2	Độ trễ lệnh điều khiển	P95 khứ hồi ≤ 1 000 ms; ACK quá hạn 10 s	P2
NFR3	Độ tin cậy dữ liệu	Mất bản tin QoS1 = 0 % ở tải danh định	P3, P7
NFR4	Phát hiện offline	≤ 30 s	P4, F7
NFR5	Chịu lỗi mạng	Không mất dữ liệu khi mất mạng ≤ 120 chu kỳ đo; tự kết nối lại	P4
NFR6	Chống trùng	Không lưu bản ghi trùng khi gửi lại	P5
NFR7	Tài nguyên	Backend ≤ 512 MB RAM, chạy trên laptop 8 GB	P6
NFR8	Bảo mật	Đáp ứng toàn bộ mục 4.5 đề bài [1]; TLS cho mọi đường truyền qua LAN	S1–S9
NFR9	Khả năng mở rộng	Xử lý ≥ 50 thiết bị đồng thời ở 1 bản tin/s	P8
NFR10	Khả dụng triển khai	Toàn hệ thống chạy bằng 1 lệnh Docker Compose	Chương 11

2.5. Các ràng buộc

Bảng 2.4. *Ràng buộc thiết kế*

Ràng buộc	Nội dung và cách đáp ứng

Ràng buộc	Nội dung và cách đáp ứng
Chi phí	Linh kiện phổ thông, giá rẻ, mua được tại cửa hàng điện tử trong nước; phần mềm hoàn toàn mã nguồn mở, không thuê dịch vụ cloud. Tổng chi phí phần cứng: 538.000 vnđ
Thiết bị	ESP32 chỉ có 520 KB SRAM, module WROOM-32 có 4 MB flash [4]: firmware dùng bộ đệm tĩnh, JSON nhỏ (≤ 1 KB). Kết quả biên dịch: dùng khoảng 15 % RAM và 74 % flash. ADC2 không dùng được khi bật WiFi nên MQ-2 phải nối ADC1 (GPIO34) [5].
Năng lượng	Node cấp điện liên tục qua USB 5 V. MQ-2 có sợi đốt cần nung liên tục (công suất sợi đốt ≤ 800 mW, khoảng 150 mA ở 5 V [6]), nên không áp dụng deep sleep; ưu tiên độ sẵn sàng hơn tiết kiệm năng lượng. Tổng dòng đỉnh ~0,6 A nên dùng nguồn ≥ 1 A (mục 5.5).
Mạng	WiFi 2,4 GHz trong LAN, không có IP công cộng; có thể mất sóng ngắn hạn. Giao thức phải nhẹ, có QoS, phát hiện mất kết nối và đồng bộ lại.
Độ trễ	Báo động tại chỗ trong 1 chu kỳ đo (mặc định 5 s), không phụ thuộc mạng; cảnh báo trên web dưới 1 s sau khi bản tin đến.
Độ tin cậy	Không mất dữ liệu khi backend hoặc broker khởi động lại (phiên MQTT bền, QoS1); thiết bị tự khởi động lại nếu mất WiFi quá 5 phút.
An toàn	Chỉ dùng điện áp thấp 5 V/3,3 V, không đấu điện lưới 220 V; quạt 12 V chạy ở 5 V qua relay; cảm biến không thay thế đầu báo đạt chuẩn.
Quyền riêng tư	Chỉ thu thập dữ liệu tối thiểu (tên đăng nhập, họ tên, email cảnh báo tùy chọn, IP trong audit log); không thu thập vị trí, hình ảnh, âm thanh.

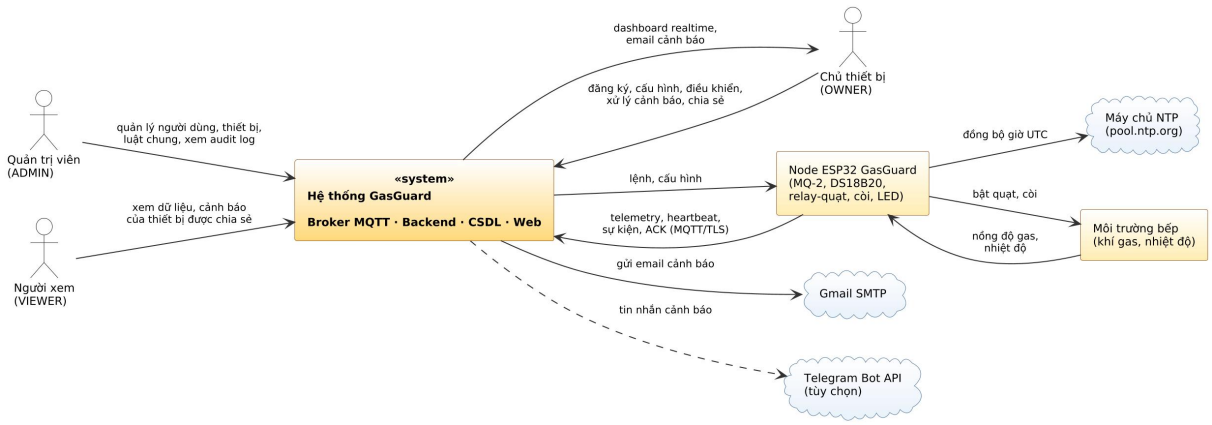
2.6. Tiêu chí đánh giá thành công

- Thiết bị vật lý chạy đầu-cuối: đo → broker → backend → dashboard và lệnh ngược lại có ACK, demo đủ 10 bước của mục 8.4 đề bài.
- Đạt toàn bộ chỉ tiêu NFR1–NFR8 trong Bảng 2.3; NFR9 đạt ở mức 50 thiết bị.
- Vượt qua ít nhất 2 kiểm thử bảo mật (nhóm đặt mục tiêu 9 ca S1–S9).
- Mã nguồn chạy được trên máy mới chỉ với .env và lệnh Docker Compose; không có bí mật trong repository.

CHƯƠNG 3. PHÂN TÍCH VÀ LỰA CHỌN GIẢI PHÁP

3.1. Sơ đồ bối cảnh hệ thống

Sơ đồ bối cảnh (Hình 3.1) coi GasGuard là một hộp đen và chỉ ra các thực thể bên ngoài trao đổi dữ liệu với hệ thống: ba nhóm người dùng, node ESP32 đặt tại bếp (tương tác với môi trường vật lý), máy chủ NTP để đồng bộ giờ cho timestamp, dịch vụ Gmail SMTP để gửi email cảnh báo và Telegram Bot API (tùy chọn, đã cài đặt nhưng không dùng mặc định vì không ổn định ở mạng trong nước).



Hình 3.1. *Sơ đồ bối cảnh hệ thống GasGuard*

3.2. Lựa chọn vi điều khiển và cảm biến

Bảng 3.1. *So sánh và lựa chọn phần cứng*

Thành phần	Phương án cân nhắc	Lựa chọn và lý do
Vi điều khiển	ESP8266; Arduino Uno + module WiFi; Raspberry Pi	ESP32-WROOM-32: WiFi tích hợp, CPU 2 nhân 240 MHz, 520 KB SRAM đủ cho TLS và bộ đệm 120 bản đo, nhiều kênh ADC 12 bit [4]. ESP8266 chỉ có 1 ADC và RAM hạn chế cho TLS; Raspberry Pi tốn điện, không có ADC, khởi động chậm.
Cảm biến khí	MQ-5, MQ-6 (chuyên LPG); cảm biến điện hóa	MQ-2: nhạy với LPG, propan, khói trong dải 300–10 000 ppm, module có sẵn chân AO, giá rẻ, phổ biến [6]. MQ-6 chọn lọc LPG tốt hơn nhưng khó mua; cảm biến điện hóa đắt và không phù hợp khí cháy.
Cảm biến nhiệt	DHT11, DHT22, LM35, thermistor	DS18B20 đầu inox: số hóa trên bus 1-Wire nên không chịu nhiễu analog, sai số ±0,5 °C trong dải −10…+85 °C, dải đo −55…+125 °C, đầu inox chịu nhiệt gần bếp [7]. Nhiệt độ tăng cao là dấu hiệu cháy đi kèm rò rỉ.
Cơ cấu chấp hành	MOSFET điều khiển quạt; servo đóng van	Relay 5 V + quạt cách ly mạch điều khiển với tải; còi chủ động qua 2N2222 vì GPIO không cấp đủ dòng. Không làm van tự động vì cần thiết bị đạt chuẩn an toàn gas.

3.3. Lựa chọn giao thức

Nhóm so sánh ba giao thức phổ biến cho thiết bị theo các tiêu chí đề bài yêu cầu thiết kế (xác nhận, online/offline, QoS, chiều xuống) [8], [2]:

Bảng 3.2. *So sánh giao thức cho chiều thiết bị ↔ máy chủ*

Tiêu chí	HTTP/REST	CoAP	MQTT (chọn)
Mô hình	Yêu cầu – phản hồi, thiết bị phải chủ động hỏi lệnh (polling)	Yêu cầu – phản hồi trên UDP, có cơ chế Observe [9]	Publish/subscribe qua broker, lệnh đẩy xuống ngay
Đảm bảo giao nhận	Tự cài đặt retry	Confirmable message	QoS 0/1/2 có sẵn [10]
Phát hiện offline	Tự cài heartbeat	Tự cài	Keep-alive + Last Will (LWT) do broker phát

Tiêu chí	HTTP/REST	CoAP	MQTT (chọn)
Giữ trạng thái cuối	Không	Không	Retained message (cấu hình, trạng thái)
Bảo mật	HTTPS	DTLS	TLS + tài khoản, ACL theo topic
Hỗ trợ trên ESP32	Tốt	Hạn chế	Tốt (thư viện MQTT, hỗ trợ TLS)

MQTT được chọn vì đáp ứng sẵn các yêu cầu bắt buộc của mục 4.3 đề bài: QoS1 đảm bảo bản tin tới ít nhất một lần, LWT phát hiện mất kết nối đột ngột, retained message giữ cấu hình mới nhất cho thiết bị khi kết nối lại, và phiên bền (persistent session) giữ hàng đợi bản tin khi backend khởi động lại [10]. HTTP/REST chỉ dùng giữa trình duyệt và backend; WebSocket với giao thức con STOMP dùng để đẩy dữ liệu realtime lên giao diện [11], [12].

3.4. Lựa chọn broker, backend, cơ sở dữ liệu và giao diện

Bảng 3.3. *Lựa chọn nền tảng phần mềm*

Thành phần	Lựa chọn và lý do
Broker	Eclipse Mosquitto 2.0 mã nguồn mở, rất nhẹ (đo được dưới 4 MiB RAM với 100 thiết bị) [13]. Plugin Dynamic Security cho phép backend tạo, khóa, xóa tài khoản và ACL của từng thiết bị lúc đang chạy qua topic $CONTROL/dynamic-security/v1, có hiệu lực ngay [14]. Đây là điểm then chốt để đáp ứng yêu cầu "mỗi thiết bị một thông tin xác thực" và "thu hồi quyền kết nối".
Backend	Spring Boot 3.5, Java 21: Spring Security cho JWT và phân quyền theo phương thức, Spring Data JPA, Flyway quản lý phiên bản schema, WebSocket STOMP có sẵn, mô hình sự kiện nội bộ để tách tiếp nhận dữ liệu khỏi cảnh báo và realtime [15], [16].
Cơ sở dữ liệu	PostgreSQL 16: dữ liệu quan hệ (người dùng, thiết bị, quyền) và chuỗi thời gian nằm trong cùng một CSDL; ràng buộc UNIQUE kết hợp INSERT … ON CONFLICT DO NOTHING chống trùng ở mức CSDL [17]; kiểu JSONB cho cấu hình và trạng thái cơ cấu chấp hành. Với quy mô hàng chục thiết bị, CSDL chuỗi thời gian riêng (InfluxDB) chưa cần thiết.
Giao diện	React 18 + Vite, Recharts cho biểu đồ, @stomp/stompjs cho realtime; phục vụ qua nginx có HTTPS.
Triển khai	Docker Compose chạy 4 container bằng một lệnh, giống nhau trên mọi máy; bí mật chỉ đến từ .env [18].

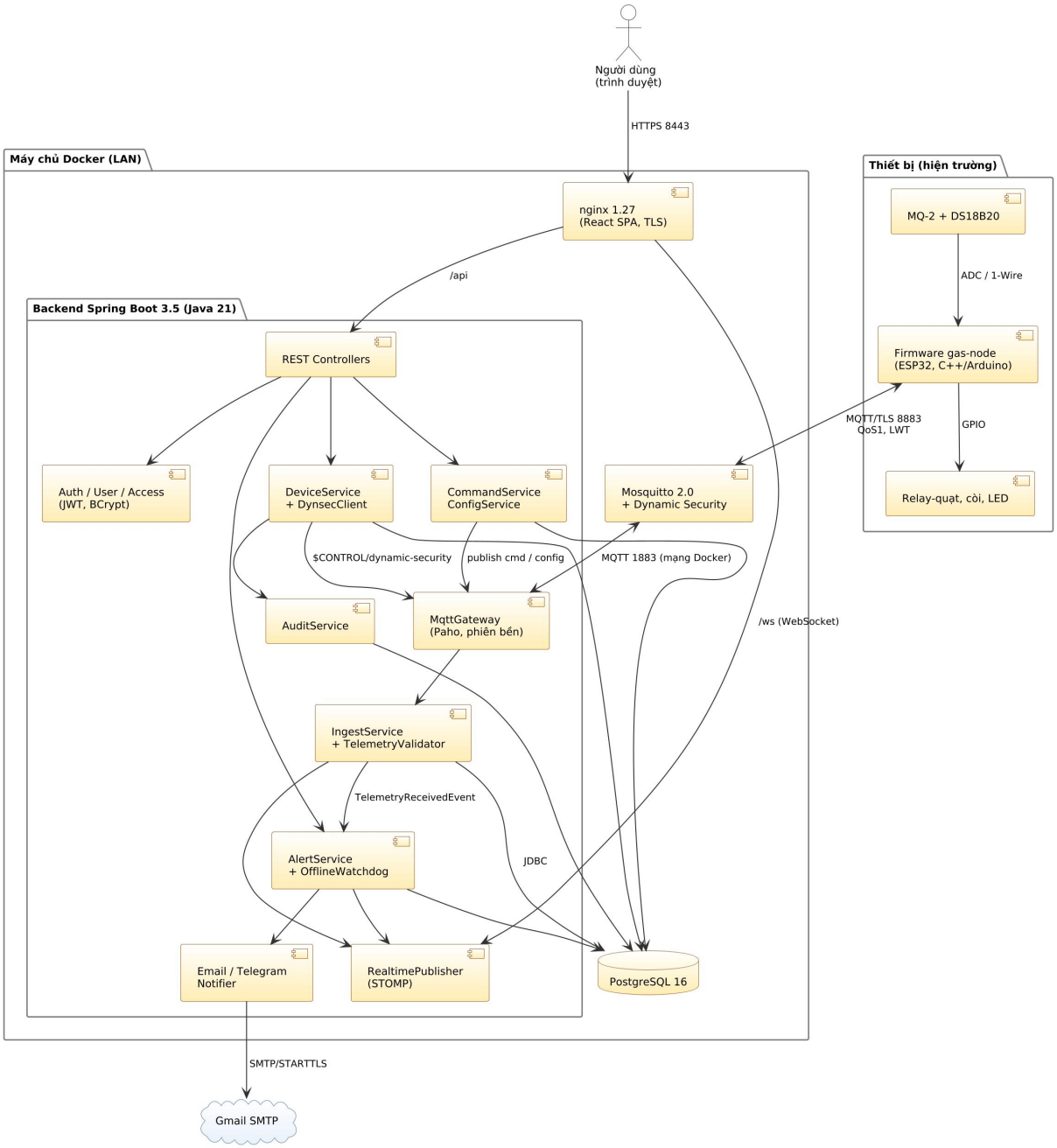
3.5. Vì sao không dùng nền tảng IoT có sẵn

Các nền tảng như Blynk, ThingSpeak hay Adafruit IO giúp hiển thị dữ liệu nhanh, nhưng không cho phép tự thiết kế vòng đời thiết bị, cấp và thu hồi thông tin xác thực riêng cho từng thiết bị, phân quyền theo chủ sở hữu và chia sẻ, hay ghi audit log theo yêu cầu. Đề bài cũng không chấp nhận sản phẩm chỉ dựa trên dashboard có sẵn [1]. Vì vậy nhóm tự xây dựng toàn bộ backend và chỉ dùng các thành phần hạ tầng mã nguồn mở (broker, CSDL, web server).

CHƯƠNG 4. KIẾN TRÚC HỆ THỐNG

4.1. Kiến trúc tổng thể

Hình 4.1 trình bày kiến trúc thành phần. Thiết bị chỉ kết nối tới broker; backend là client MQTT duy nhất có quyền đọc mọi topic thiết bị và gửi lệnh. Trình duyệt chỉ nói chuyện với nginx: tài nguyên tĩnh của SPA, REST (/api) và WebSocket (/ws) được chuyển tới backend. Bên trong backend, các thành phần giao tiếp với nhau qua sự kiện (TelemetryReceivedEvent, AlertChangedEvent, CommandUpdatedEvent…), nhờ đó luồng tiếp nhận dữ liệu không phụ thuộc vào việc gửi email hay đẩy realtime.

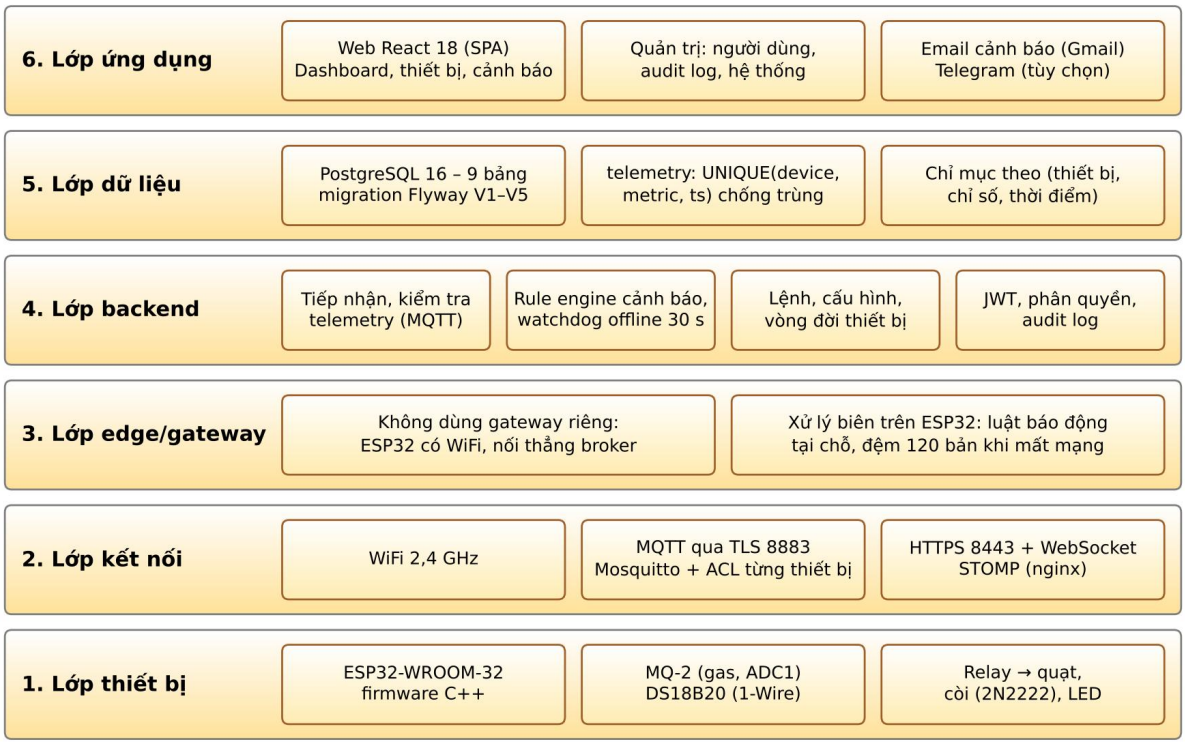


Hình 4.1. *Sơ đồ kiến trúc tổng thể (sơ đồ thành phần)*

4.2. Kiến trúc phân lớp

Hình 4.2 ánh xạ hệ thống vào sáu lớp theo yêu cầu đề bài. Hệ thống **không có gateway riêng**: ESP32 có WiFi và đủ tài nguyên chạy TLS nên kết nối thẳng tới

broker, giảm một điểm hỏng và chi phí. Chức năng của lớp edge được đặt ngay trên thiết bị: luật báo động tại chỗ, xử lý nhiễu và đệm dữ liệu khi mất mạng.



Hình 4.2. *Sơ đồ các lớp của hệ thống*

4.3. Luồng dữ liệu đầu-cuối

Chiều lên (thiết bị → người dùng): cảm biến → ESP32 (đọc, lọc, quy đổi, luật tại chỗ) → MQTT/TLS telemetry QoS1 → Mosquitto (kiểm tra ACL) → backend (kiểm tra hợp lệ, chống trùng, lưu CSDL) → sự kiện → rule engine cảnh báo và RealtimePublisher → WebSocket → dashboard; song song, cảnh báo được gửi email.

Chiều xuống (người dùng → thiết bị): giao diện → REST POST /commands (JWT) → backend kiểm tra quyền, lưu lệnh → MQTT cmd QoS1 → ESP32 thực hiện (relay, còi) → cmd/ack → backend cập nhật kết quả, độ trễ → phản hồi về giao diện. Cấu hình đi theo luồng tương tự với topic config (retained) và config/ack. Chi tiết từng bước được trình bày bằng sequence diagram ở Chương 6 và Chương 8.

CHƯƠNG 5. THIẾT KẾ PHẦN CỨNG

5.1. Danh sách linh kiện

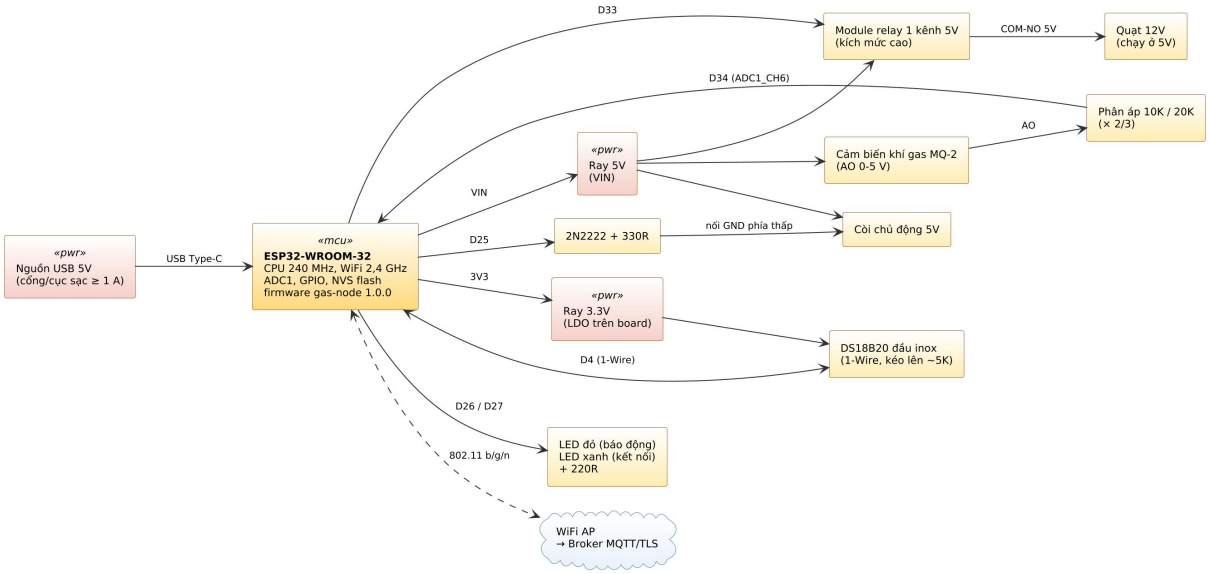
Bảng 5.1. *Danh sách linh kiện (BOM)*

Linh kiện	SL	Chức năng
ESP32-WROOM-32 DevKit, cổng Type-C + cáp USB	1	Vi điều khiển, WiFi, nguồn 5 V (VIN) và 3,3 V
Breadboard MB-102 830 lỗ	1	Lắp mạch không hàn
Module cảm biến gas MQ-2	1	Đo nồng độ khí gas (ngõ ra analog AO)

Linh kiện	SL	Chức năng
Cảm biến nhiệt DS18B20 dạng dây, đầu inox	1	Đo nhiệt độ (1-Wire)
LED đỏ, LED xanh lá	1 + 1	Báo động / trạng thái kết nối
Transistor NPN 2N2222	1	Khóa điện tử cho còi
Còi chủ động (active buzzer) 5 V	1	Báo động âm thanh
Module relay 1 kênh 5 V (kích mức cao)	1	Đóng/ngắt quạt
Quạt 12 V (chạy ở 5 V)	1	Quạt hút mô phỏng
Điện trở 220 Ω / 330 Ω / 10 kΩ / 20 kΩ	2 / 1 / 3 / 1	Hạn dòng LED, base transistor, phân áp, kéo lên 1-Wire
Dây cắm đực–cái, đực–đực	~15 + ~15	Kết nối

5.2. Sơ đồ khối phần cứng

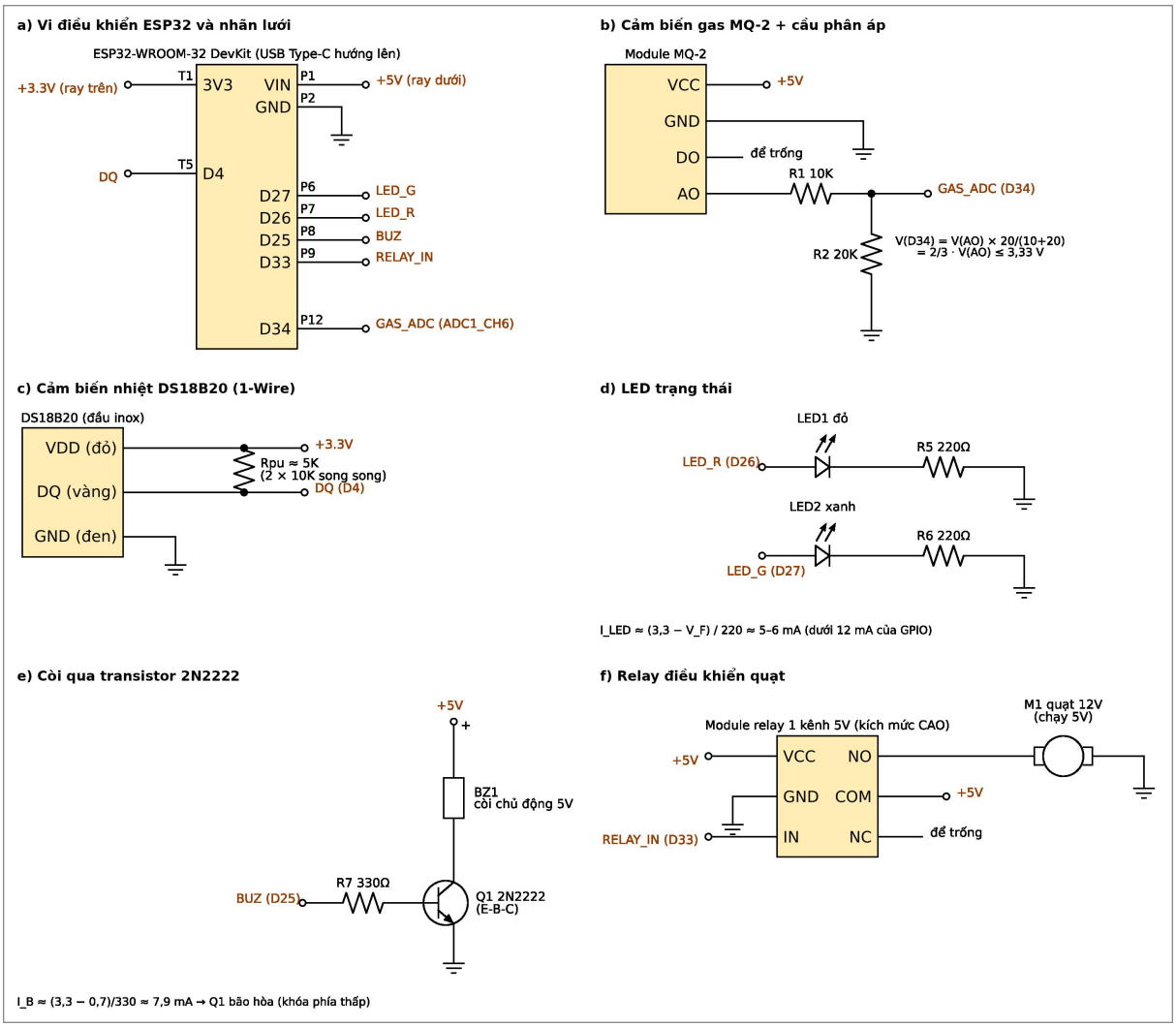
Hình 5.1 thể hiện các khối và đường tín hiệu. Nguồn USB 5 V cấp cho ESP32; chân VIN đưa 5 V ra ray dưới của breadboard để nuôi các tải 5 V (MQ-2, relay, còi), còn chân 3V3 (từ bộ ổn áp trên board) đưa ra ray trên để nuôi DS18B20. Hai ray nguồn tách biệt, hai ray GND nối chung.



Hình 5.1. *Sơ đồ khối phần cứng của node GasGuard*

5.3. Sơ đồ nguyên lý và bảng chân

Sơ đồ nguyên lý (Hình 5.2) được vẽ theo nhãn lưới: các điểm cùng tên (+5V, +3.3V, GAS_ADC, DQ, RELAY_IN, BUZ, LED_R, LED_G) được nối với nhau. Bảng 5.2 đối chiếu chân ESP32 với hằng số trong firmware/gas-node/config.h.



Hình 5.2. *Sơ đồ nguyên lý kết nối cảm biến và cơ cấu chấp hành*

Bảng 5.2. *Bảng chân ESP32 (đếm từ phía cổng USB xuống)*

Vị trí	Chân	Chức năng	Chiều	Hằng số firmware
Phải-1	VIN (5 V)	Nguồn 5 V ra ray dưới	–	–
Phải-2	GND	Mass chung	–	–
Phải-6	D27	LED xanh (trạng thái kết nối)	Ra	PIN_LED_GREEN 27
Phải-7	D26	LED đỏ (báo động, lỗi cảm biến)	Ra	PIN_LED_RED 26
Phải-8	D25	Còi qua 2N2222	Ra	PIN_BUZZER 25
Phải-9	D33	Relay → quạt	Ra	PIN_RELAY 33
Phải-12	D34	MQ-2 AO qua phân áp (ADC1_CH6)	Vào	PIN_MQ2_AO 34
Trái-1	3V3	Nguồn 3,3 V ra ray trên	–	–
Trái-5	D4	DS18B20 DATA (1-Wire)	Vào/ra	PIN_DS18B20 4

5.5. Tính toán thiết kế

a) Cầu phân áp cho MQ-2

Module MQ-2 được nuôi 5 V nên điện áp AO có thể lên tới gần 5 V, trong khi chân ADC của ESP32 chỉ chịu tối đa 3,3 V [4]. Cầu phân áp R1 = 10 kΩ, R2 = 20 kΩ

cho V(D34) = V(AO) × 20/(10 + 20) = 2/3 · V(AO) ≤ 3,33 V. Firmware nhân ngược hệ số 1,5 (MQ2_DIVIDER_RATIO). Tổng trở phân áp 30 kΩ lớn hơn nhiều điện trở tải RL ≈ 1 kΩ của module nên không làm sai lệch đáng kể phép đo. ADC của ESP32 phi tuyến ở hai đầu dải đo; firmware dùng analogReadMilliVolts() (áp dụng hệ số hiệu chuẩn ghi trong eFuse) và lấy trung bình 16 mẫu để giảm nhiễu do quạt dùng chung nguồn 5 V [5].

b) Quy đổi nồng độ ppm

Điện trở cảm biến được tính từ điện áp ra: Rs = RL · (Vc − Vout)/Vout. Sau 60 s làm nóng, ở lần chạy đầu firmware lấy trung bình 50 mẫu trong không khí sạch và tính R0 = Rs/9,83 (tỉ số Rs/R0 trong không khí sạch theo datasheet), lưu vào bộ nhớ NVS. Nồng độ LPG được tính theo dạng hàm mũ của đường đặc tuyến trên thang log–log trong datasheet: ppm = A · (Rs/R0)^B, với A = 574,25 và B = −2,222 là hệ số nội suy từ đường cong LPG [6]. Do sai số chế tạo của MQ-2 và ảnh hưởng của nhiệt độ, độ ẩm, giá trị ppm chỉ mang tính ước lượng; hệ thống dùng nó để so ngưỡng tương đối chứ không thay thế thiết bị đo chuẩn.

c) Điện trở kéo lên 1-Wire, LED và transistor

- **DS18B20:** hai điện trở 10 kΩ song song cho Rpu = 5 kΩ, gần giá trị 4,7 kΩ khuyến nghị cho bus 1-Wire [7], [19]. Cảm biến nuôi 3,3 V nên mức logic DATA tương thích trực tiếp với ESP32.
- **LED:** I = (3,3 − V_F)/220 Ω ≈ 5–6 mA với LED đỏ (V_F ≈ 2 V), dưới mức dòng khuyến nghị của GPIO.
- **Còi:** dòng base I_B ≈ (3,3 − 0,7)/330 ≈ 7,9 mA; với h_FE tối thiểu khoảng 35–50, transistor đủ bão hòa để dẫn dòng còi vài chục mA. Còi đặt phía collector (khóa phía thấp) nên chân GPIO 3,3 V vẫn điều khiển được tải 5 V. Một số lô 2N2222/PN2222 có thứ tự chân khác nhau, cần kiểm tra bằng đồng hồ trước khi lắp.
- **Relay:** module kích **mức cao** (D33 = HIGH thì đóng), tương ứng RELAY_ACTIVE_LOW 0; khi khởi động ESP32 giữ D33 ở mức thấp nên quạt không tự chạy.

d) Ngân sách công suất

Bảng 5.3. *Ước tính dòng tiêu thụ trên nguồn 5 V*

Tải	Dòng ước tính	Ghi chú
ESP32 (WiFi phát)	80–250 mA	Đỉnh khi truyền [4]
MQ-2 (sợi đốt)	~150 mA	Liên tục [6]
Cuộn relay	~70 mA	Khi quạt bật
Quạt 12 V chạy 5 V	~50–150 mA	Mặc dù nguồn thấp hơn đề xuất của quạt nhưng quạt vẫn quay ở tốc độ ổn, đủ để demo
Còi + 2 LED	~40 mA	Khi báo động
Tổng đỉnh	~0,6 A	Vượt 500 mA của cổng USB 2.0

Vì tổng dòng đỉnh có thể vượt khả năng cổng USB 2.0, nhóm dùng cổng hoặc cục sạc **≥ 1 A**; nếu không, ESP32 có thể bị sụt áp và khởi động lại (brownout) đúng lúc relay đóng. Hướng cải tiến: thêm tụ 100–470 µF trên ray 5 V và diode dập ngược song song với quạt.

5.6. Thiết kế firmware

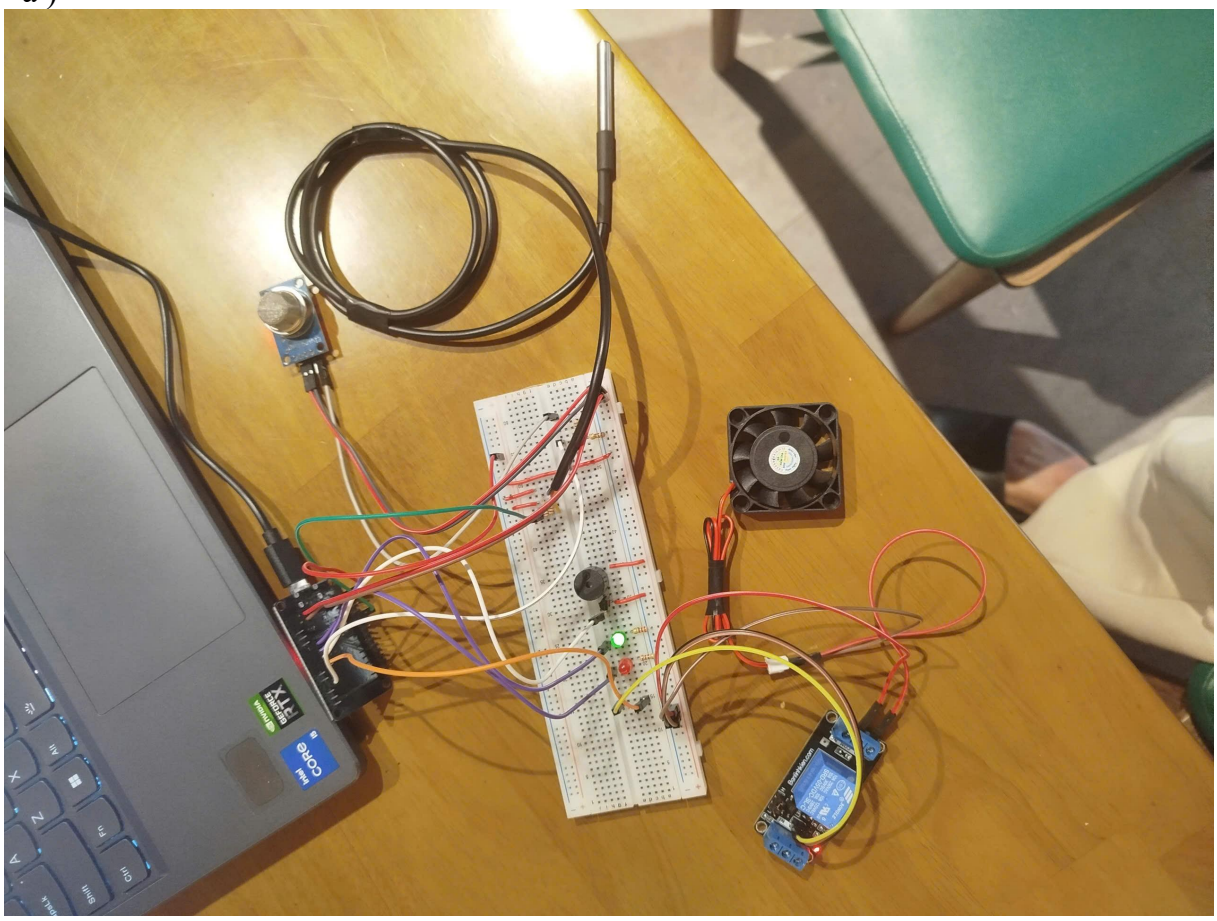
Firmware viết bằng C++ trên lõi Arduino-ESP32 3.x, gồm gas-node.ino (820 dòng), config.h (chân, ngưỡng, tham số) và secrets.h (WiFi, tài khoản MQTT, không commit). Vòng lặp chính không chặn (sơ đồ hoạt động ở Hình C.1, Phụ lục C) thực hiện: duy trì kết nối, xử lý lệnh/cấu hình, đo theo chu kỳ, áp dụng luật tại chỗ, gửi hoặc đệm dữ liệu, gửi sự kiện và heartbeat, cập nhật đèn báo.

Các cơ chế đáng chú ý:

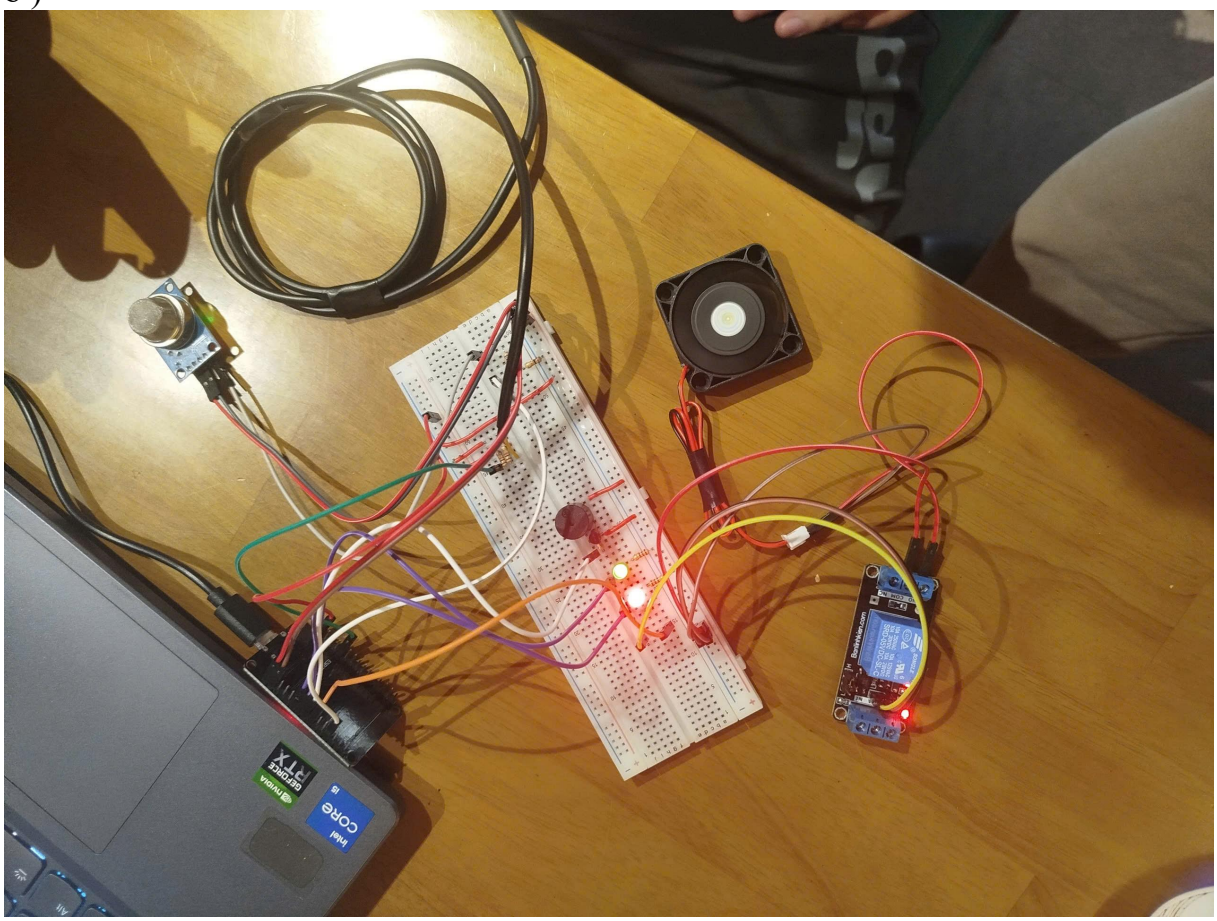
- **Luật tại biên có trễ (hysteresis):** báo động gas khi vượt ngưỡng cấu hình (mặc định 400 ppm), chỉ tắt khi xuống dưới 90 % ngưỡng; báo động nhiệt bật ở 45 °C, tắt ở 42 °C. Khi báo động: LED đỏ, còi kêu ngắt quãng và (nếu autoFan) tự bật quạt; hết báo động 60 s mới tắt quạt để hút nốt khí còn lại. Luật chạy độc lập với mạng.
- **Chế độ tay/tự động:** lệnh FAN_SET/BUZZER_SET chuyển sang MANUAL và giữ trạng thái người dùng chọn, FAN_AUTO/BUZZER_AUTO trả về tự động. Chế độ được lưu vào NVS nên giữ nguyên sau khi khởi động lại.
- **Phát hiện lỗi cảm biến:** DS18B20 trả −127 °C (mất kết nối), 85 °C lúc mới cấp nguồn hoặc giá trị ngoài dải; MQ-2 có điện áp ≈ 0 V. Lỗi 3 lần liên tiếp thì gửi sự kiện SENSOR_FAULT, hồi phục thì gửi SENSOR_OK; LED đỏ nháy.
- **Mất mạng:** bản đo được đưa vào bộ đệm vòng 120 phần tử (10 phút ở chu kỳ 5 s), gửi bù đúng thứ tự khi kết nối lại; nếu đầy thì bỏ bản cũ nhất và đếm số bản bị bỏ. Mất WiFi quá 5 phút thì khởi động lại.
- **Thời gian:** chỉ gửi bản đo khi đã đồng bộ NTP (UTC, mili giây) vì TLS cần giờ đúng để kiểm tra hạn chứng chỉ và backend từ chối timestamp sai.

5.7. Hình ảnh nguyên mẫu

a)



b)



c)

```
[  1077.5s] Kết nối MQTT (TLS) 192.168.1.245:8884 với tài khoản esp32-001 ...
[  1078.1s] Đã kết nối broker
[  1078.1s] Gửi bù 1 bản đo lưu lúc mất mạng (còn 0)
[  1082.5s] gas=1.7 ppm  temp=28.25 C  quạt=TẮT  còi=im(tay)
[  1087.6s] gas=1.7 ppm  temp=28.25 C  quạt=TẮT  còi=im(tay)
[  1092.6s] gas=1.6 ppm  temp=28.25 C  quạt=TẮT  còi=im(tay)
[  1097.7s] gas=9.3 ppm  temp=28.25 C  quạt=TẮT  còi=im(tay)
[  1102.7s] Gas 3909.4 ppm > ngưỡng 400 -> TỰ BẬT QUẠT
[  1102.7s] gas=3909.4 ppm  temp=28.25 C  quạt=BẬT  còi=im(tay)
[  1107.8s] gas=265.7 ppm  temp=28.25 C  quạt=BẬT  còi=im(tay)
[  1112.8s] gas=40.3 ppm  temp=28.25 C  quạt=BẬT  còi=im(tay)
[  1117.9s] gas=25.1 ppm  temp=28.25 C  quạt=BẬT  còi=im(tay)
```

Hình 5.3. *Nguyên mẫu node GasGuard: (a) mạch đã lắp ráp; (b) khi báo động – LED đỏ, quạt chạy; (c) Serial Monitor*

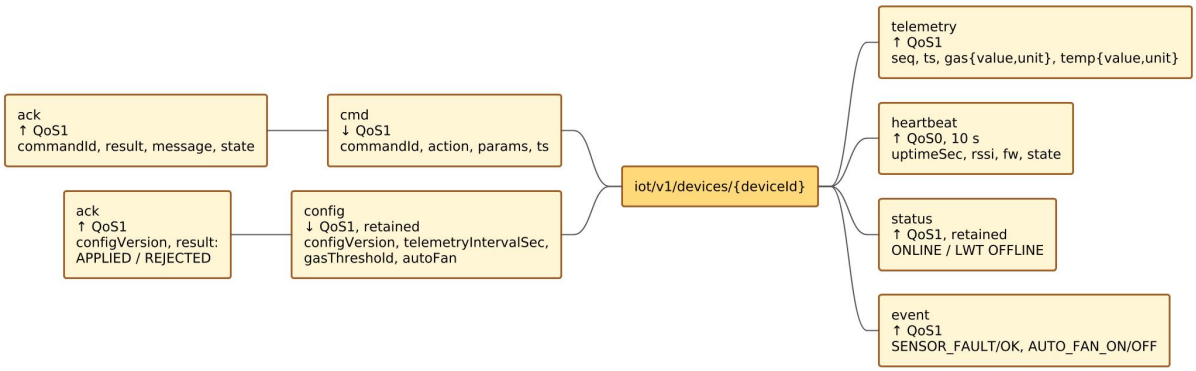
5.8. An toàn điện

Toàn bộ mạch dùng điện áp thấp 5 V và 3,3 V lấy từ USB, không đấu nối điện lưới 220 V. Khi thử cảm biến, nhóm chỉ xả một lượng nhỏ khí từ bật lửa gas (không đánh lửa) ở nơi thoáng khí, không dùng bình gas thật. Thiết bị là nguyên mẫu học tập, không thay thế đầu báo gas đạt chuẩn.

CHƯƠNG 6. THIẾT KẾ GIAO THỨC VÀ LUỒNG DỮ LIỆU

6.1. Cấu trúc topic MQTT

Mọi topic có gốc iot/v1/devices/{deviceId}/…. Tiền tố v1 cho phép thay đổi định dạng sau này mà không ảnh hưởng thiết bị cũ. deviceId là mã thiết bị, đồng thời là username và clientId MQTT, nên broker có thể giới hạn mỗi thiết bị chỉ publish/subscribe trong nhánh của chính nó.



Hình 6.1. *Cây topic MQTT (↑ thiết bị gửi, ↓ backend gửi)*

Bảng 6.1. *Danh sách topic, chiều, QoS và mục đích*

Topic	Chiều	QoS / retained	Mục đích
telemetry	Thiết bị → backend	1	Số đo gas, nhiệt độ
heartbeat	Thiết bị → backend	0	10 s/lần: uptime, RSSI, firmware, trạng thái quạt/còi

Topic	Chiều	QoS / retained	Mục đích
status	Thiết bị → backend	1, retained	ONLINE khi kết nối; LWT OFFLINE do broker phát khi mất kết nối đột ngột
event	Thiết bị → backend	1	SENSOR_FAULT, SENSOR_OK, AUTO_FAN_ON, AUTO_FAN_OFF
cmd	Backend → thiết bị	1	Lệnh FAN_SET, FAN_AUTO, BUZZER_SET, BUZZER_AUTO, PING
cmd/ack	Thiết bị → backend	1	Kết quả lệnh và trạng thái cơ cấu chấp hành
config	Backend → thiết bị	1, retained	Cấu hình có số phiên bản
config/ack	Thiết bị → backend	1	APPLIED hoặc REJECTED kèm lý do

6.2. Định dạng bản tin

Bản tin dùng JSON UTF-8. Mỗi bản tin mang deviceId, timestamp ts là **epoch mili giây UTC** lấy từ NTP, và mỗi chỉ số mang **đơn vị tường minh** (ppm, C). seq tăng dần giúp backend ước lượng tỷ lệ mất bản tin.

```
// telemetry (QoS1)
{"deviceId":"esp32-001","seq":1024,"ts":1794021884000,"fw":"1.0.0",
 "gas":{"value":64.0,"unit":"ppm"},"temp":{"value":29.5,"unit":"C"}}

// heartbeat (QoS0)
{"deviceId":"esp32-001","ts":1794021890000,"uptimeSec":3605,"rssi":-55,"fw":"1.0.0",
 "state":{"fan":false,"fanMode":"AUTO","buzzer":false,"buzzerMode":"AUTO"}}

// status (QoS1, retained) – LWT có cùng dạng với "state":"OFFLINE"
{"deviceId":"esp32-001","state":"ONLINE","ts":1794021800000,"fw":"1.0.0",
 "hw":"esp32-devkit-mq2-ds18b20","ip":"192.168.1.23"}

// cmd (QoS1) → cmd/ack (QoS1)
{"commandId":"5f0c…-uuid","action":"FAN_SET","params":{"on":true},"ts":1794021900000}
{"commandId":"5f0c…-uuid","result":"SUCCESS","message":"Quạt BẬT (giữ bật đến khi đổi lại)",
 "state":{"fan":true,"fanMode":"MANUAL","buzzer":false,"buzzerMode":"AUTO"},"ts":…}

// config (QoS1, retained) → config/ack (QoS1)
{"configVersion":3,"telemetryIntervalSec":5,"gasThreshold":400,"autoFan":true}
{"configVersion":3,"result":"APPLIED","message":"OK","applied":{…},"ts":…}
```

6.3. QoS, thử lại, quá hạn và chống trùng

- **QoS1** cho telemetry, sự kiện, lệnh, cấu hình và ACK: bản tin được gửi lại tới khi nhận PUBACK, đảm bảo "ít nhất một lần" [10]. Heartbeat dùng **QoS0** vì mất một heartbeat không ảnh hưởng (backend chỉ đánh OFFLINE sau 30 s im lặng).
- **Phiên bền ở backend:** backend kết nối với cleanSession=false và clientId cố định nên khi backend khởi động lại, broker giữ đăng ký và xếp hàng các bản tin QoS1; backend chỉ trả PUBACK sau khi đã xử lý xong bản tin.
- **Chống trùng dữ liệu:** QoS1 và cơ chế gửi bù có thể tạo bản tin trùng. CSDL có ràng buộc UNIQUE(device_id, metric, ts) và backend dùng INSERT … ON

CONFLICT DO NOTHING [17]; bản trùng chỉ tăng bộ đếm duplicateReadingsIgnored.

- **Chống trùng lệnh:** commandId (UUID) là khóa chính bảng commands; firmware nhớ ACK của các lệnh gần đây và gửi lại ACK cũ nếu nhận lại cùng commandId thay vì thực hiện lần nữa.
- **Quá hạn lệnh:** lệnh ở trạng thái SENT quá 10 s không có ACK được tác vụ định kỳ chuyển sang TIMEOUT; ACK đến muộn vẫn được ghi nhận.
- **Cấu hình có phiên bản:** configVersion tăng mỗi lần đổi; thiết bị trả config/ack với đúng phiên bản; backend so config_version với config_applied_version để biết thiết bị đã áp dụng chưa (PENDING, APPLIED, REJECTED). Topic config là retained nên thiết bị nhận cấu hình mới nhất ngay khi kết nối lại.

6.4. Phát hiện online/offline

Hệ thống dùng hai cơ chế bổ sung cho nhau. (1) **LWT**: khi kết nối, thiết bị đăng ký Last Will status = OFFLINE (retained); nếu thiết bị mất kết nối đột ngột, broker tự phát bản tin này sau khi hết keep-alive (cấu hình 15 s, broker chờ tối đa 1,5 lần) [10]. (2) **Watchdog** ở backend chạy mỗi 5 s, đánh OFFLINE thiết bị không gửi bất kỳ bản tin nào trong 30 s (bằng 3 chu kỳ heartbeat), phòng trường hợp LWT không đến. Khi chuyển OFFLINE, rule engine mở cảnh báo loại OFFLINE; khi thiết bị gửi lại status = ONLINE hoặc bất kỳ bản tin hợp lệ nào, trạng thái trở lại ONLINE.

6.5. Kiểm tra hợp lệ dữ liệu đầu vào

Bảng 6.2. *Quy tắc kiểm tra telemetry ở backend*

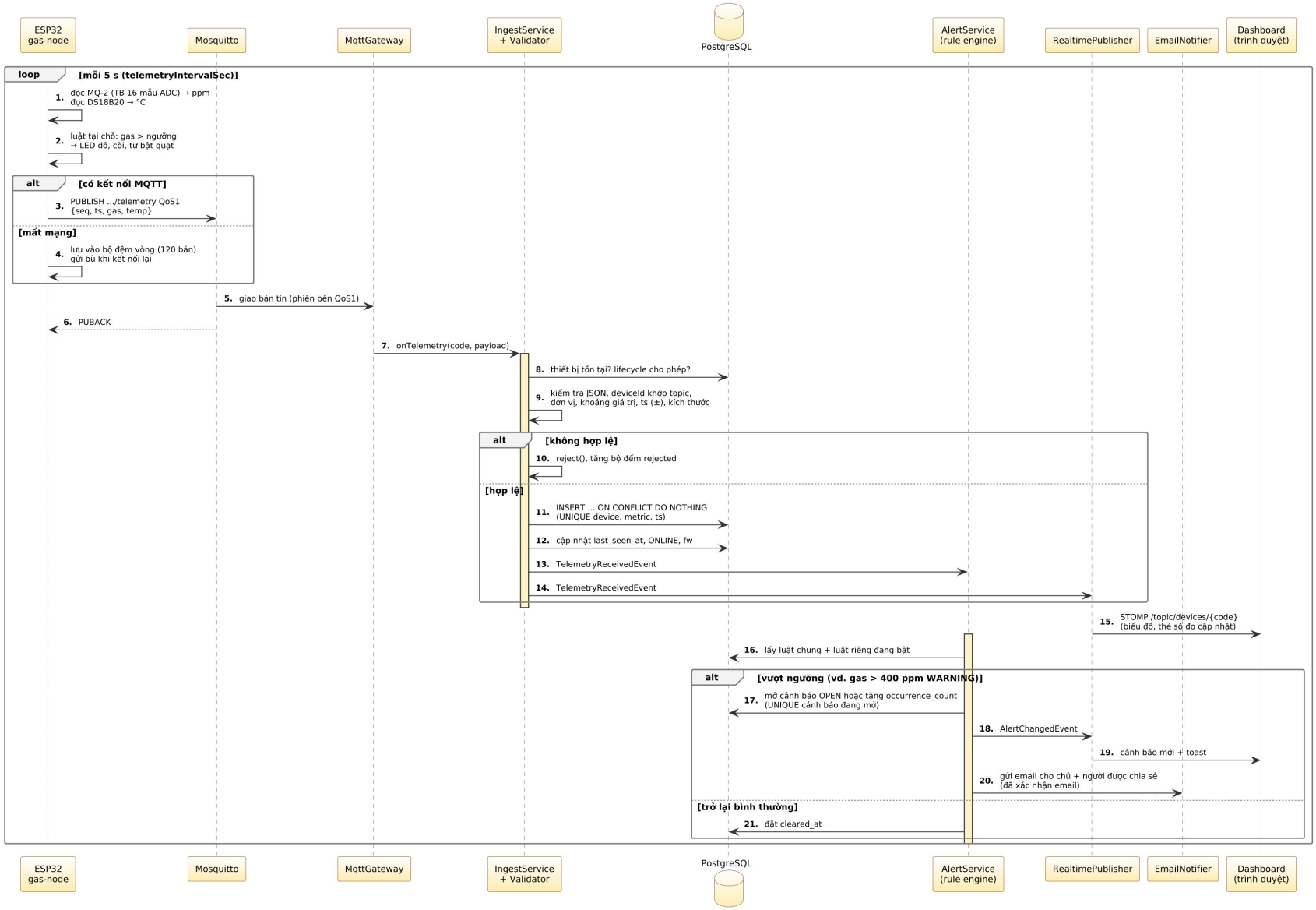
Kiểm tra	Quy tắc (mã lỗi)
Thiết bị	Mã thiết bị tồn tại và trạng thái vòng đời cho phép nhận dữ liệu; deviceId trong payload khớp topic (DEVICE_ID_MISMATCH)
Cấu trúc	JSON hợp lệ (MALFORMED_JSON), kích thước ≤ 4 096 byte (PAYLOAD_TOO_LARGE), không có trường lạ (UNKNOWN_FIELD), có ít nhất một chỉ số (NO_METRIC)
Thời gian	ts là số nguyên epoch ms (INVALID_TS); sau năm 2024 (TS_NOT_SYNCED); không vượt quá hiện tại 2 phút (TS_IN_FUTURE); không cũ hơn 7 ngày (TS_TOO_OLD)
Đơn vị và khoảng	gas: ppm, 0–10 000; temp: C, −55…125 (WRONG_UNIT, OUT_OF_RANGE)
Khác	seq là số nguyên ≥ 0; fw là chuỗi 1–30 ký tự

Bản tin không hợp lệ bị bỏ qua, tăng bộ đếm theo lý do (hiển thị ở trang Hệ thống) và không ảnh hưởng các bản tin khác.

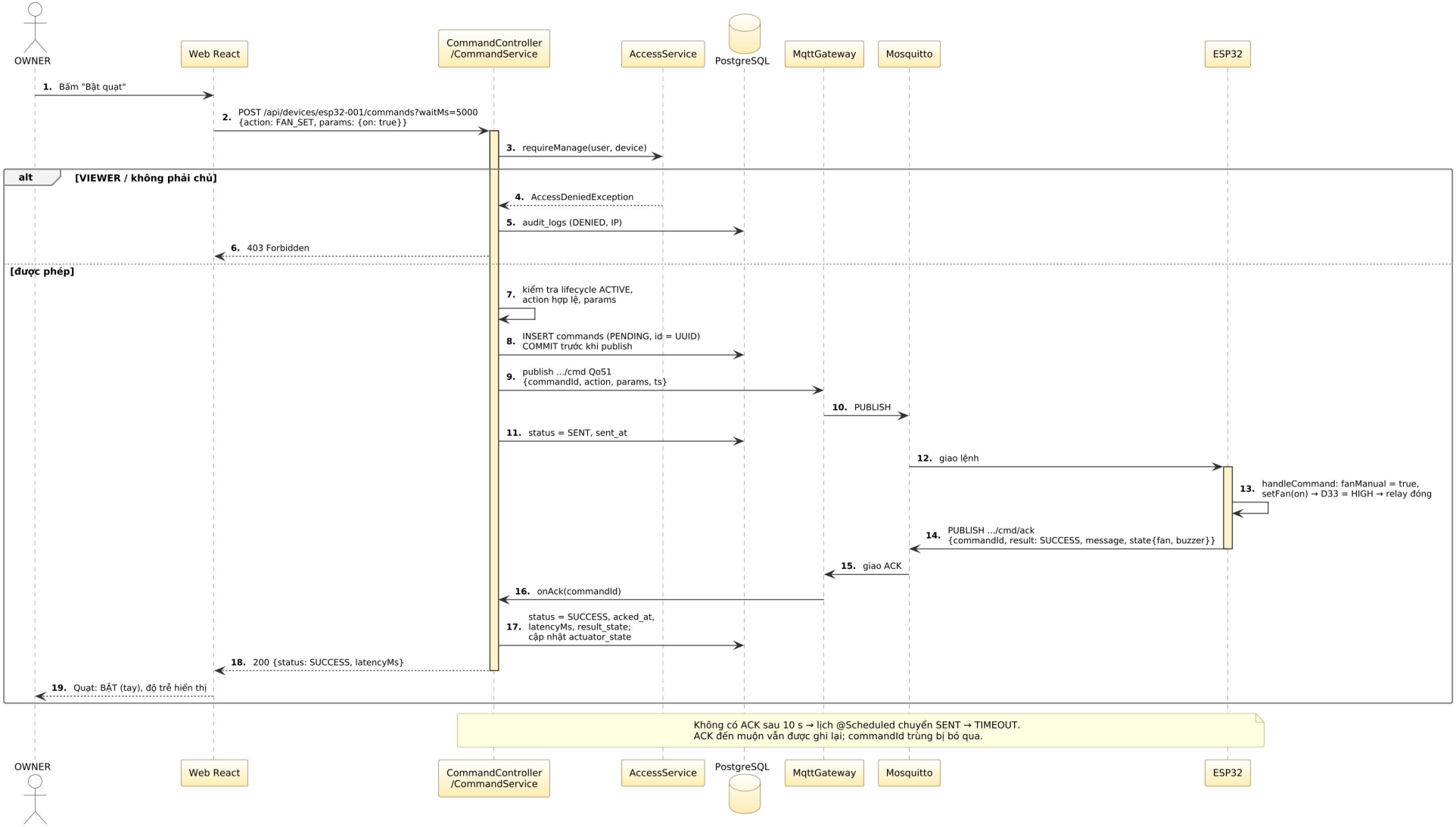
6.6. Sequence diagram các quy trình chính

Hình 6.2 mô tả quy trình **thu thập dữ liệu và sinh cảnh báo**, từ lúc ESP32 đọc cảm biến tới khi dashboard cập nhật và email được gửi. Hình 6.3 mô tả quy trình **gửi**

lệnh điều khiển và nhận xác nhận, gồm cả nhánh bị từ chối do không đủ quyền. Hai quy trình còn lại (đăng ký – kích hoạt và thu hồi thiết bị) được trình bày ở Chương 8.



Hình 6.2. *Sequence diagram: thu thập telemetry, cập nhật dashboard và sinh cảnh báo*

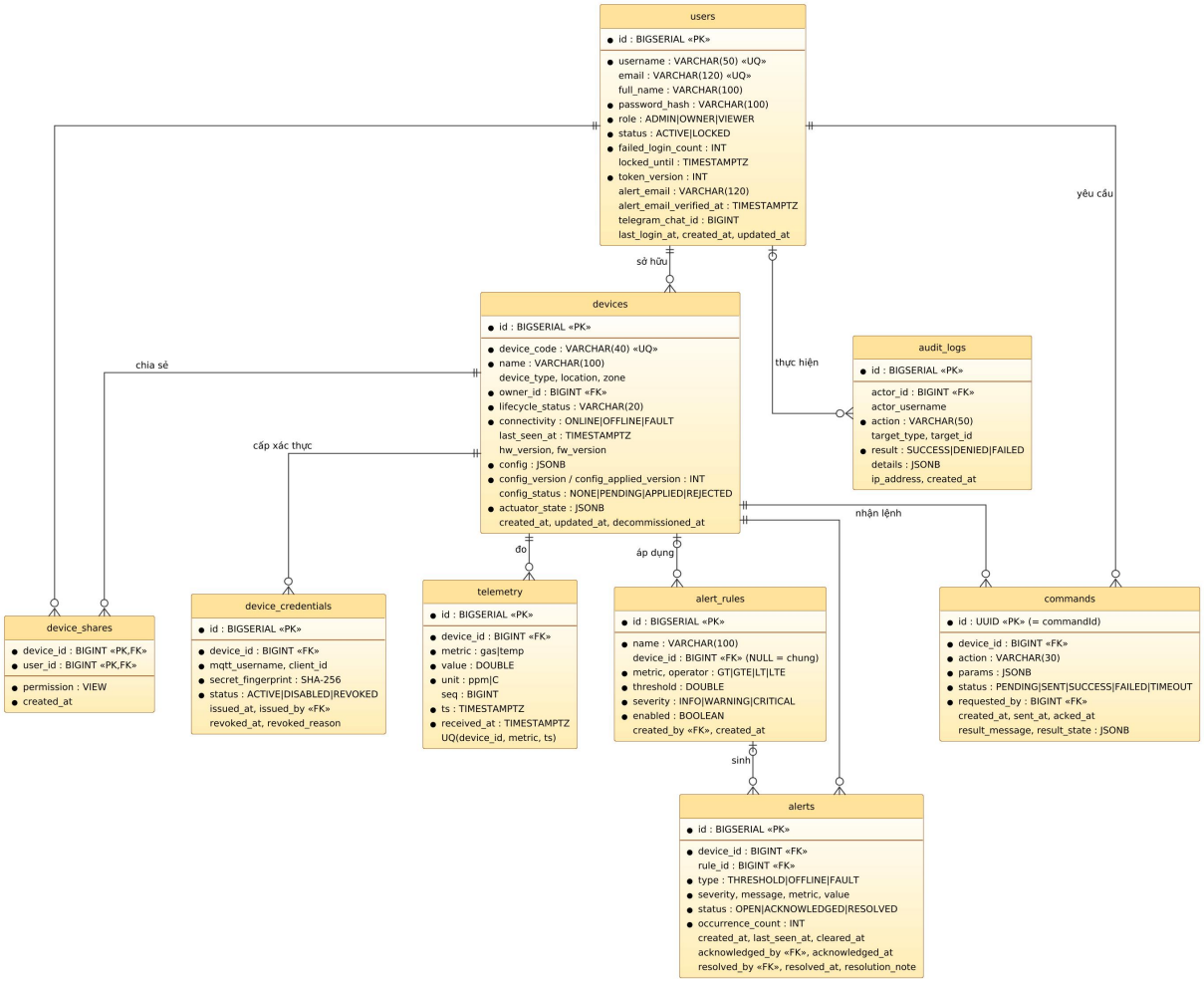


Hình 6.3. *Sequence diagram: gửi lệnh điều khiển quạt và nhận ACK*

CHƯƠNG 7. THIẾT KẾ CƠ SỞ DỮ LIỆU, API VÀ BACKEND

7.1. Mô hình dữ liệu (ERD)

CSDL gồm 9 bảng, được tạo và nâng cấp bằng 5 migration Flyway (V1–V5) nên schema luôn khớp với mã nguồn; Hibernate chạy ở chế độ validate, không tự sửa schema. Hình 7.1 trình bày ERD.



Hình 7.1. *Mô hình thực thể – liên kết (ERD) của GasGuard*

Bảng 7.1. *Mô tả các bảng và quyết định thiết kế*

Bảng	Nội dung và quyết định thiết kế
users	Tài khoản, vai trò, trạng thái. password_hash dùng BCrypt; failed_login_count, locked_until phục vụ khóa tạm; token_version tăng khi đăng xuất/đổi mật khẩu để thu hồi mọi JWT cũ; email cảnh báo phải xác nhận (alert_email_verified_at).
devices	Mã thiết bị duy nhất (ràng buộc định dạng ^[a-z0-9][a-z0-9-]{2,39}$), chủ sở hữu, vị trí, khu vực, lifecycle_status và connectivity tách riêng; phiên bản phần cứng/firmware; cấu hình JSONB có config_version, config_applied_version, config_status; actuator_state JSONB đóng vai trò device shadow.
device_shares	Khóa chính kép (thiết bị, người dùng), quyền VIEW.
device_credentials	Lịch sử thông tin xác thực MQTT. Chỉ lưu SHA-256 fingerprint, không lưu mật khẩu; chỉ mục duy nhất có điều kiện bảo đảm mỗi thiết bị có tối đa một thông tin xác thực chưa thu hồi.

Bảng	Nội dung và quyết định thiết kế
telemetry	Mỗi chỉ số một dòng (metric, value, unit); lưu cả ts (thời điểm đo trên thiết bị) và received_at (thời điểm backend nhận) để đo độ trễ; UNIQUE(device_id, metric, ts) chống trùng; chỉ mục (device_id, metric, ts DESC) cho truy vấn theo khoảng thời gian.
alert_rules	Luật ngưỡng: device_id NULL là luật chung. 3 luật mặc định: gas > 400 ppm (WARNING), gas > 1 000 ppm (CRITICAL), nhiệt > 50 °C (WARNING).
alerts	Loại THRESHOLD/OFFLINE/FAULT, mức độ, trạng thái OPEN → ACKNOWLEDGED → RESOLVED, người xác nhận/xử lý. Chỉ mục duy nhất có điều kiện trên (thiết bị, loại, luật) khi chưa RESOLVED gộp các lần vi phạm lặp lại vào một cảnh báo (occurrence_count) thay vì sinh hàng loạt.
commands	Khóa chính là commandId (UUID) gửi xuống thiết bị; trạng thái PENDING → SENT → SUCCESS/FAILED/TIMEOUT; lưu thời điểm gửi, ACK và trạng thái trả về.
audit_logs	Ai, làm gì, trên đối tượng nào, kết quả SUCCESS/DENIED/FAILED, chi tiết JSONB và địa chỉ IP.

7.2. Kiến trúc backend

Bảng 7.2. *Các gói chính của backend*

Gói	Trách nhiệm
mqtt	MqttGateway (Eclipse Paho, phiên bền, tự kết nối lại với thời gian chờ tăng dần), MqttTopics, DynsecClient gọi API Dynamic Security của broker.
ingest	DeviceMessageRouter phân luồng theo topic; IngestService xử lý telemetry, heartbeat, status, event; TelemetryValidator; IngestStats thống kê thông lượng, độ trễ, số bản tin bị từ chối.
alert	AlertService (rule engine), AlertRuleService, OfflineWatchdog.
service	Nghiệp vụ: DeviceService, DeviceLifecycle (máy trạng thái), CommandService, ConfigService, AuthService, UserService, AccessService (kiểm tra quyền trên thiết bị), AuditService, DashboardService.
security	JwtService, JwtAuthFilter, SecurityConfig (BCrypt cost 12, quy tắc theo URL), xử lý lỗi 401/403 thống nhất.
realtime	Cấu hình WebSocket STOMP, StompAuthInterceptor (xác thực JWT khi CONNECT, kiểm tra quyền khi SUBSCRIBE), RealtimePublisher.
notify	Gửi cảnh báo qua email (SMTP STARTTLS) và Telegram (tùy chọn).
web	13 REST controller, DTO có Bean Validation, GlobalExceptionHandler trả lỗi dạng JSON thống nhất.

7.3. Mô tả API

API theo phong cách REST, dữ liệu JSON, xác thực bằng header Authorization: Bearer <JWT> (trừ đăng nhập và đăng ký). Tổng cộng **53 endpoint**; Bảng 7.3 tóm tắt theo nhóm, danh sách đầy đủ ở Phụ lục A. Mã lỗi thống nhất: 400 dữ liệu sai, 401 chưa xác thực/token sai, 403 không đủ quyền, 404 không tồn tại, 409 chuyển trạng thái không hợp lệ, 423 tài khoản bị khóa tạm.

Bảng 7.3. *Tóm tắt API theo nhóm chức năng*

Nhóm / tiền tố	Số EP	Chức năng chính
/api/auth	6	Đăng ký, đăng nhập, đăng xuất, xem/sửa hồ sơ, đổi mật khẩu
/api/auth/me/email-alerts, /telegram	9	Đăng ký, xác nhận, thử, hủy kênh nhận cảnh báo
/api/admin/users	5	Danh sách, tạo, đổi vai trò, khóa, mở khóa (ADMIN)
/api/devices	11	Danh sách, đăng ký, chi tiết, cập nhật, cấp/đổi xác thực, lịch sử xác thực, vòng đời, chia sẻ
/api/devices/{code}/telemetry	5	Số đo mới nhất, dữ liệu thô phân trang, chuỗi cho biểu đồ, thống kê, xuất CSV
/api/devices/{code}/commands, /config	5	Gửi lệnh (có chờ ACK), lịch sử, chi tiết lệnh; xem/đổi cấu hình
/api/alerts, /api/alert-rules	8	Tra cứu, tổng hợp, xác nhận, xử lý cảnh báo; CRUD luật
/api/dashboard, /api/system, audit	4	Tổng quan dashboard, thống kê tiếp nhận, audit log, lịch sử hoạt động thiết bị

Ví dụ gửi lệnh và kết quả trả về khi có ACK trong thời gian chờ:

```
POST /api/devices/esp32-001/commands?waitMs=5000
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
{"action":"FAN_SET","params":{"on":true}}

200 OK
{"id":"5f0c…","action":"FAN_SET","status":"SUCCESS","latencyMs":53,
 "resultMessage":"Quạt BẬT (giữ bật đến khi đổi lại)","resultState":{"fan":true,"fanMode":"MANUAL",…}}
```

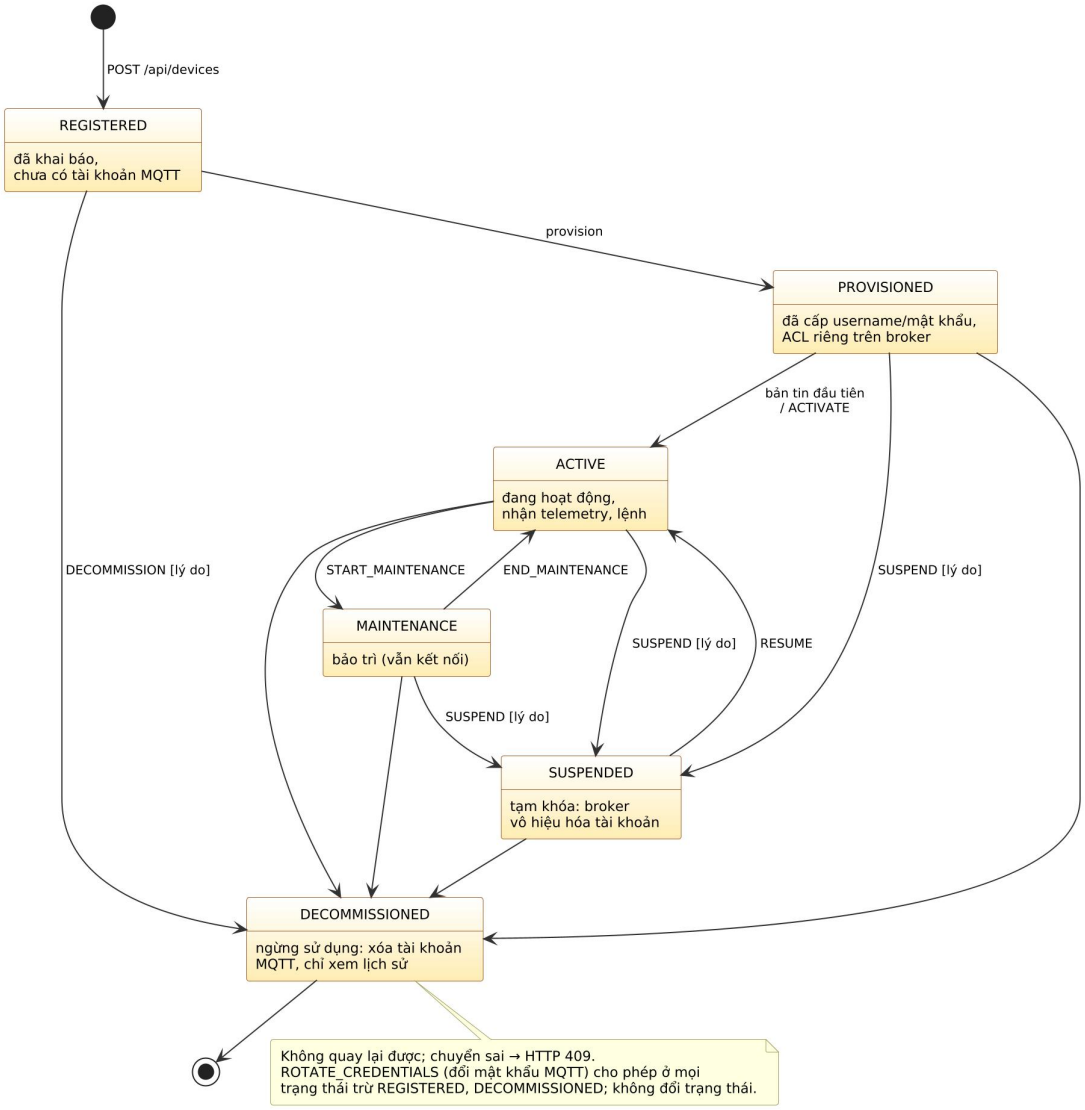
7.4. Rule engine cảnh báo

Mỗi khi có TelemetryReceivedEvent, AlertService lấy các luật đang bật áp dụng cho thiết bị (luật chung và luật riêng), so sánh từng chỉ số theo toán tử GT/GTE/LT/LTE. Vi phạm lần đầu mở cảnh báo OPEN; vi phạm tiếp theo khi cảnh báo còn mở chỉ tăng occurrence_count và last_seen_at; khi giá trị trở lại bình thường, cảnh báo được đánh dấu cleared_at nhưng vẫn chờ người dùng xử lý. Sự kiện OFFLINE (từ LWT hoặc watchdog) và SENSOR_FAULT tạo cảnh báo loại tương ứng. Mỗi thay đổi được đẩy realtime và gửi email cho chủ thiết bị cùng người được chia sẻ đã xác nhận email; mỗi email gửi riêng từng người để không lộ địa chỉ của nhau.

CHƯƠNG 8. QUẢN LÝ VÒNG ĐỜI THIẾT BỊ

8.1. Mô hình trạng thái

Vòng đời thiết bị là một máy trạng thái hữu hạn cài đặt trong lớp DeviceLifecycle (Hình 8.1). Trạng thái kết nối (ONLINE/OFFLINE/FAULT) được tách thành một trường riêng do hệ thống tự cập nhật, vì một thiết bị ACTIVE vẫn có thể tạm mất mạng; gộp hai khái niệm vào một trạng thái sẽ làm mất thông tin quản trị. Mọi chuyển trạng thái không có trong bảng quy tắc bị từ chối với HTTP 409; tạm khóa và ngừng sử dụng bắt buộc ghi lý do.



Hình 8.1. *Mô hình trạng thái vòng đời thiết bị*

Bảng 8.1. *Ý nghĩa trạng thái và tác động lên broker*

Trạng thái	Ý nghĩa	Tác động lên broker MQTT
REGISTERED	Đã khai báo mã, tên, vị trí, chủ sở hữu	Chưa có tài khoản
PROVISIONED	Đã cấp thông tin xác thực, chờ thiết bị kết nối	Tạo client + role ACL riêng
ACTIVE	Hoạt động bình thường; tự chuyển từ PROVISIONED khi nhận bản tin hợp lệ đầu tiên	Client được bật
MAINTENANCE	Đang bảo trì, vẫn nhận dữ liệu để kiểm tra	Client được bật
SUSPENDED	Tạm khóa (nghi bị xâm nhập, mất thiết bị…)	disableClient: ngắt kết nối ngay, từ chối kết nối lại
DECOMMISSIONED	Ngừng sử dụng vĩnh viễn, chỉ xem dữ liệu lịch sử	deleteClient, deleteRole; credential REVOKED

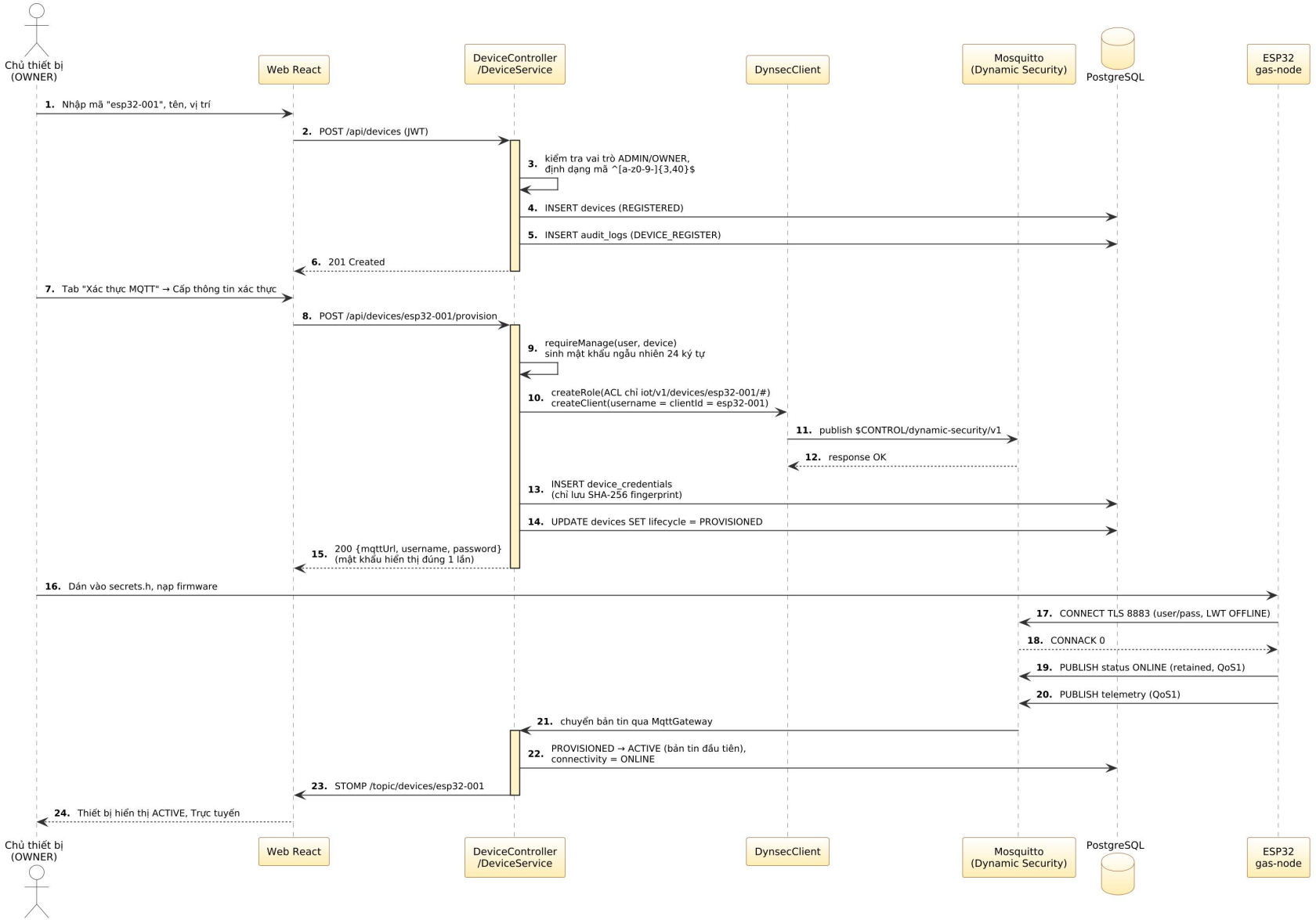
8.2. Quy trình đăng ký, cấp xác thực và kích hoạt

Hình 8.2 trình bày quy trình onboarding. Mật khẩu thiết bị (24 ký tự ngẫu nhiên) chỉ hiển thị **một lần** cho người cấp; backend chỉ lưu fingerprint SHA-256. Username

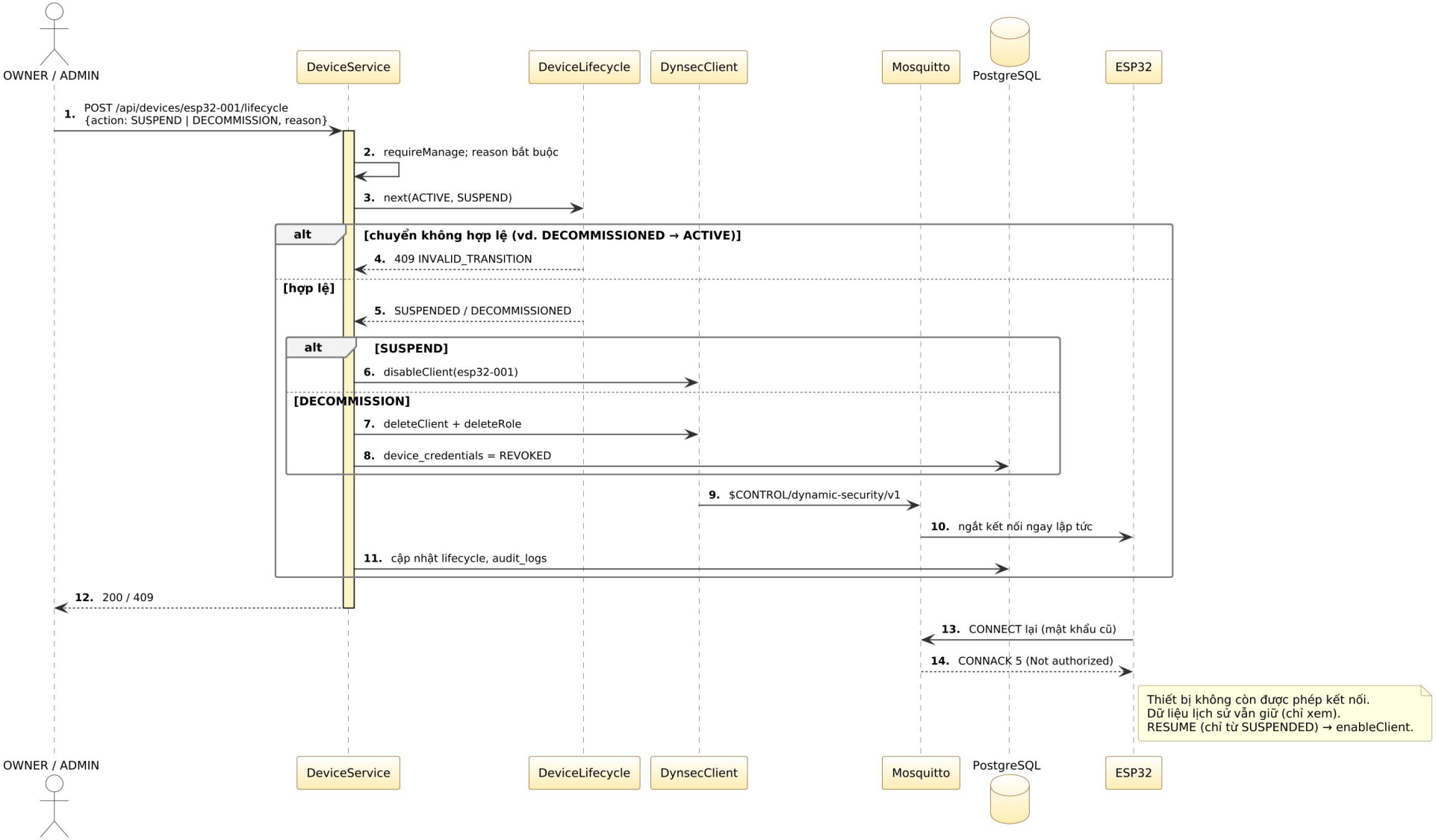
và clientId đều bằng mã thiết bị, ACL chỉ cho phép thiết bị publish/subscribe trong iot/v1/devices/{mã}/#.

8.3. Quy trình tạm khóa và ngừng sử dụng

Khi tạm khóa hoặc ngừng sử dụng, backend gọi Dynamic Security để vô hiệu hóa hoặc xóa tài khoản MQTT [14]; broker ngắt kết nối thiết bị ngay và trả CONNACK mã 5 (Not authorized) cho mọi lần kết nối lại (Hình 8.3). Đây là cách hệ thống chứng minh "thiết bị không còn được phép kết nối" trong kịch bản demo số 10.



Hình 8.2. *Sequence diagram: đăng ký, cấp thông tin xác thực và kích hoạt thiết bị*



Hình 8.3. *Sequence diagram: tạm khóa / ngừng sử dụng thiết bị*

8.4. Đối chiếu yêu cầu quản lý vòng đời

Bảng 8.2. *Đối chiếu 10 yêu cầu của mục 4.4b đề bài*

Yêu cầu	Cách đáp ứng
1. Đăng ký / onboarding	POST /api/devices → REGISTERED
2. Cấp định danh, xác thực	provision, credentials/rotate; tài khoản MQTT riêng từng thiết bị
3. Gán người dùng, vị trí, khu vực	owner_id, location, zone; chia sẻ quyền xem
4. Kích hoạt	Tự động khi nhận bản tin đầu tiên, hoặc hành động ACTIVATE
5. Cấu hình từ xa	PUT /config → topic config retained, có phiên bản và config/ack
6. Theo dõi online/offline	LWT + watchdog 30 s; dashboard realtime
7. Phiên bản phần cứng, firmware	hw_version, fw_version cập nhật từ status, telemetry, heartbeat
8. Ghi nhận lỗi, bảo trì	Trạng thái kết nối FAULT, cảnh báo FAULT; trạng thái MAINTENANCE
9. Tạm khóa, thu hồi kết nối	SUSPEND (disableClient), đổi mật khẩu thiết bị (mật khẩu cũ hết hiệu lực)
10. Ngừng sử dụng	DECOMMISSION: xóa tài khoản MQTT, giữ dữ liệu lịch sử

CHƯƠNG 9. QUẢN LÝ NGƯỜI DÙNG VÀ PHÂN QUYỀN

9.1. Quản lý tài khoản

- **Đăng ký:** người tự đăng ký nhận vai trò VIEWER; ADMIN đổi vai trò thành OWNER hoặc tạo tài khoản trực tiếp. Mật khẩu 8–72 ký tự, có ít nhất một chữ cái và một chữ số (72 là giới hạn đầu vào của BCrypt) [20], [21].
- **Đăng nhập:** trả JWT ký HS256, hạn 60 phút [22]. Sai mật khẩu 5 lần liên tiếp thì tài khoản bị khóa tạm 15 phút (HTTP 423) để chống dò mật khẩu.
- **Đăng xuất, đổi mật khẩu:** tăng token_version; JWT mang phiên bản này nên mọi token phát hành trước đó lập tức hết hiệu lực, dù chưa hết hạn.
- **Trạng thái tài khoản:** ADMIN khóa/mở khóa tài khoản; tài khoản bị khóa không đăng nhập được và không đăng ký thêm kênh WebSocket.

9.2. Ma trận vai trò và quyền

Quyền trên thiết bị phụ thuộc cả vai trò lẫn quan hệ với thiết bị: **xem** khi là ADMIN, là chủ sở hữu, hoặc được chia sẻ; **quản lý** khi là ADMIN hoặc là OWNER sở hữu thiết bị. Vì vậy một OWNER được chia sẻ thiết bị của người khác cũng chỉ được xem thiết bị đó.

Bảng 9.1. *Ma trận vai trò và quyền (✓ được phép, – không được phép)*

Chức năng	ADMIN	OWNER (thiết bị của mình)	OWNER/VIEWER (được chia sẻ)	VIEWER (không được chia sẻ)
Đăng nhập, sửa hồ sơ, đổi mật khẩu, nhận email cảnh báo	✓	✓	✓	✓

Chức năng	ADMIN	OWNER (thiết bị của mình)	OWNER/VIEWER (được chia sẻ)	VIEWER (không được chia sẻ)
Xem dashboard, thông tin, dữ liệu, biểu đồ, xuất CSV	✓ (mọi thiết bị)	✓	✓	–
Xem cảnh báo, lịch sử lệnh, cấu hình, hoạt động thiết bị	✓	✓	✓	–
Đăng ký thiết bị mới	✓	✓	–	–
Sửa thông tin, cấp/đổi xác thực MQTT	✓	✓	–	–
Chuyển vòng đời (bảo trì, khóa, ngừng dùng)	✓	✓	–	–
Gửi lệnh điều khiển, đổi cấu hình	✓	✓	–	–
Xác nhận, xử lý cảnh báo	✓	✓	–	–
Chia sẻ / thu hồi chia sẻ	✓	✓	–	–
Luật cảnh báo riêng của thiết bị	✓	✓	xem	–
Luật cảnh báo chung (mọi thiết bị)	✓	xem	xem	xem
Quản lý người dùng, audit log toàn hệ thống, thống kê hệ thống	✓	–	–	–

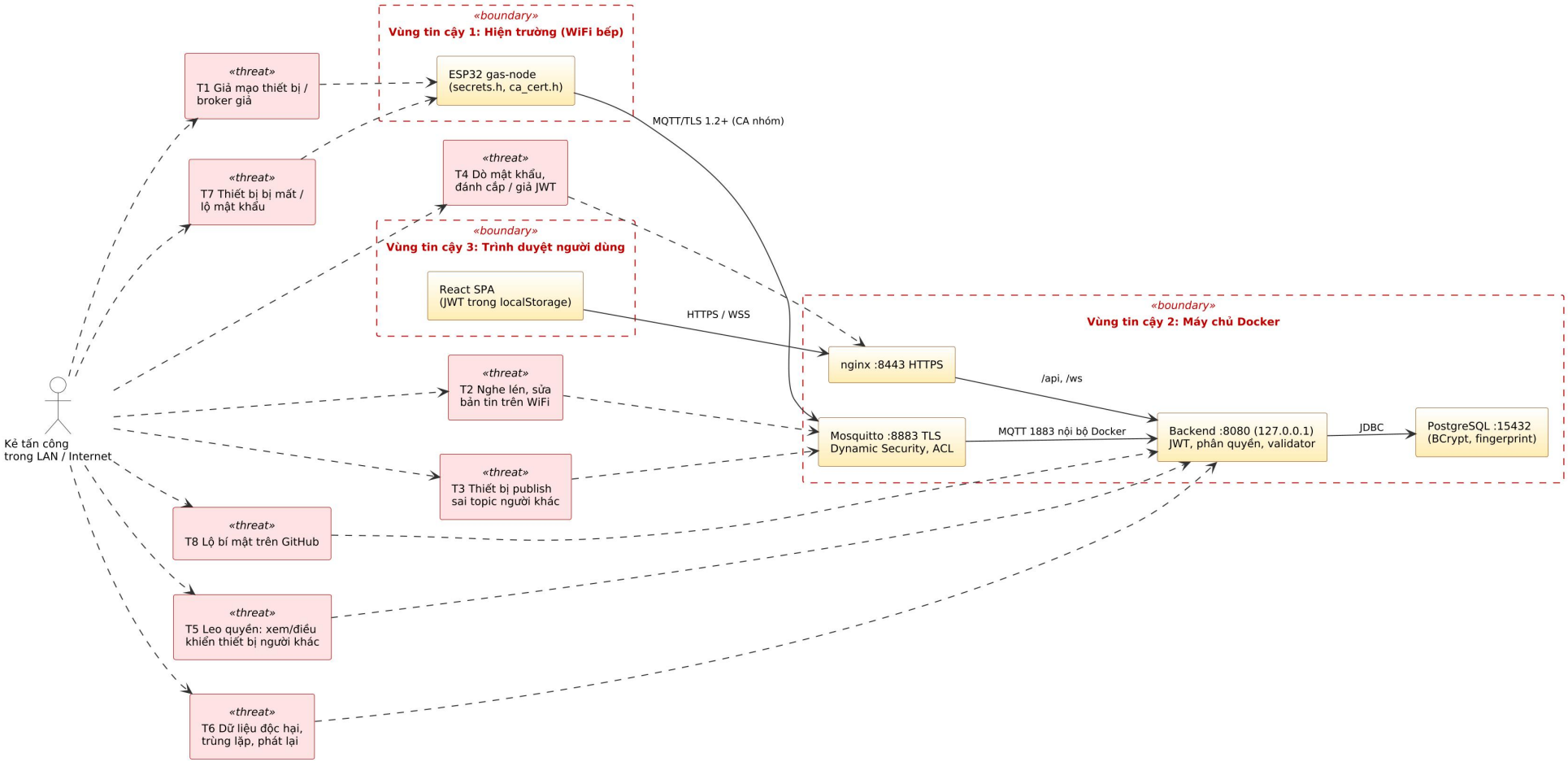
9.3. Kiểm tra quyền ở backend

Giao diện chỉ ẩn hoặc vô hiệu hóa nút để thuận tiện; quyết định cuối cùng luôn ở backend, theo khuyến nghị chống lỗi phân quyền ở mức đối tượng (BOLA) và mức chức năng của OWASP [23]. Có ba lớp kiểm tra: (1) quy tắc theo URL trong SecurityConfig (/api/admin/** chỉ ADMIN, các API khác cần xác thực); (2) @PreAuthorize trên controller quản trị; (3) AccessService.requireView/requireManage trong tầng service cho từng thiết bị cụ thể [16]. Với WebSocket, StompAuthInterceptor xác thực JWT ở khung STOMP CONNECT [12] và kiểm tra lại quyền xem thiết bị ở mỗi SUBSCRIBE, nên người dùng không nghe lén được kênh realtime của thiết bị người khác. Mọi lần bị từ chối đều được ghi audit log kèm IP.

CHƯƠNG 10. THIẾT KẾ BẢO MẬT VÀ QUYỀN RIÊNG TƯ

10.1. Mô hình mối đe dọa

Nhóm xác định ba vùng tin cậy (hiện trường, máy chủ Docker, trình duyệt) và các mối đe dọa tại ranh giới giữa chúng, phân loại theo STRIDE [24] (Hình 10.1, Bảng 10.1).



Hình 10.1. *Mô hình mối đe dọa với các vùng tin cậy*

Bảng 10.1. *Mối đe dọa chính, biện pháp kiểm soát và bằng chứng kiểm thử*

Mã	Mối đe dọa	STRIDE	Biện pháp kiểm soát	Kiểm chứng
T1	Giả mạo thiết bị hoặc broker giả	Spoofing	Mỗi thiết bị một tài khoản; ESP32 kiểm tra chứng chỉ broker bằng CA của nhóm; broker không cho kết nối ẩn danh	S4, S9
T2	Nghe lén, sửa bản tin trên WiFi	Tampering, Info. disclosure	MQTT qua TLS 1.2/1.3 cổng 8883; HTTPS 8443; cổng 1883 không mã hóa chỉ mở trong máy chủ [25]	S9
T3	Thiết bị publish/subscribe topic của thiết bị khác	Elevation	ACL Dynamic Security theo iot/v1/devices/{mã}/#; kiểm tra deviceId khớp topic	S4
T4	Dò mật khẩu, giả hoặc dùng lại JWT	Spoofing	BCrypt cost 12; khóa tạm sau 5 lần sai; JWT ký HS256 khóa ≥ 32 ký tự, hạn 60 phút, thu hồi bằng token_version	S3, S6, S8
T5	Truy cập, điều khiển thiết bị của người khác	Elevation	Kiểm tra quyền ở service cho từng thiết bị, cả REST và WebSocket; audit log khi bị từ chối	S1, S2
T6	Dữ liệu độc hại, trùng lặp, phát lại	Tampering, DoS	Validator (định dạng, đơn vị, khoảng, thời gian, kích thước ≤ 4 KB); UNIQUE chống trùng; Bean Validation cho REST	S5, P5
T7	Thiết bị bị mất hoặc lộ mật khẩu	Spoofing	Đổi mật khẩu thiết bị, tạm khóa, ngừng sử dụng → broker ngắt ngay	S7, F9
T8	Lộ bí mật qua GitHub	Info. disclosure	Bí mật chỉ trong .env, secrets.h, khóa TLS – đều trong .gitignore; repo chỉ có .env.example, secrets.h.example	Rà soát repo

10.2. Đáp ứng các yêu cầu bảo mật của đề bài

Bảng 10.2. *Đối chiếu yêu cầu bảo mật mục 4.5*

Yêu cầu	Cách làm trong GasGuard
Mật khẩu băm an toàn	BCrypt cost 12 có salt ngẫu nhiên [20]; mã xác nhận email lưu dạng SHA-256
API yêu cầu xác thực, kiểm tra quyền ở backend	JWT cho mọi API trừ đăng nhập/đăng ký; ba lớp kiểm tra quyền (mục 9.3)
Định danh và xác thực riêng cho thiết bị; không dùng chung khóa	Mỗi thiết bị một username/mật khẩu ngẫu nhiên 24 ký tự và ACL riêng
Không đưa bí mật lên GitHub; truyền bí mật qua biến môi trường	.env, secrets.h, ca_cert.h, *.key trong .gitignore; Docker Compose báo lỗi nếu thiếu biến bắt buộc
Kiểm tra, làm sạch dữ liệu đầu vào	TelemetryValidator; Bean Validation; truy vấn tham số hóa qua JPA
Khóa, thu hồi, vô hiệu hóa thiết bị	SUSPEND, ROTATE_CREDENTIALS, DECOMMISSION
Audit log	Đăng nhập, quản lý người dùng, vòng đời, lệnh, cấu hình, chia sẻ, xuất dữ liệu, truy cập bị từ chối (kèm IP)
TLS/HTTPS	TLS 1.2/1.3 cho ESP32 ↔ broker và trình duyệt ↔ web; CA riêng của nhóm vì mạng LAN không có tên miền công khai

10.3. Quyền riêng tư

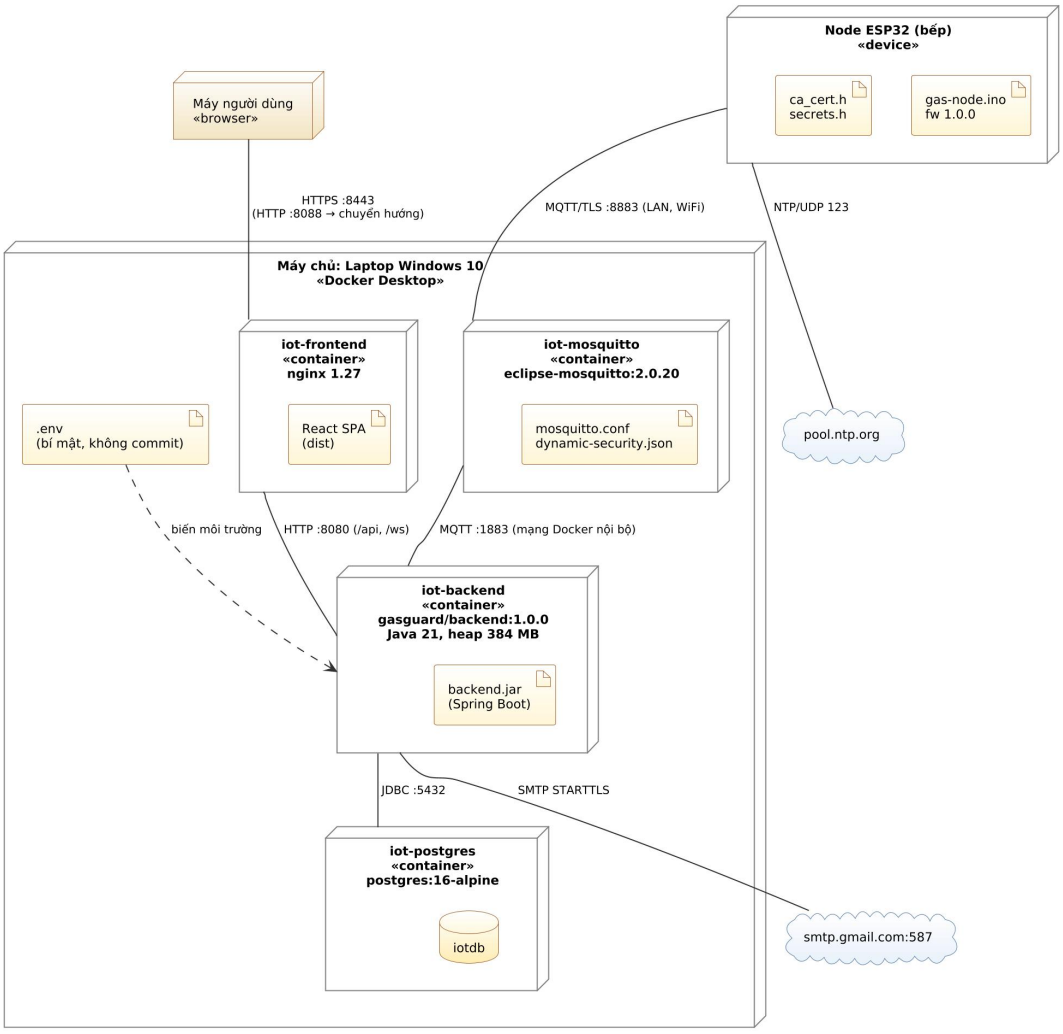
Hệ thống chỉ thu thập: tên đăng nhập, họ tên, email nhận cảnh báo (tùy chọn, phải xác nhận qua mã), địa chỉ IP trong audit log, và dữ liệu đo môi trường của thiết bị. Không thu thập vị trí GPS, hình ảnh hay âm thanh. Người dùng chỉ thấy thiết bị và dữ liệu trong phạm vi được cấp; mỗi email cảnh báo gửi riêng từng người; người dùng tự tắt nhận email bất kỳ lúc nào; log ứng dụng che bớt địa chỉ email. Dữ liệu nằm trên máy chủ trong LAN của người dùng, không gửi tới dịch vụ cloud của bên thứ ba ngoài nội dung email cảnh báo.

Hạn chế đã biết: JWT được lưu trong localStorage của trình duyệt cho đơn giản, nên dễ bị đánh cắp nếu giao diện có lỗ hổng XSS; React tự thoát ký tự khi hiển thị nên giảm rủi ro này, hướng cải tiến là dùng cookie HttpOnly. Chứng chỉ do CA tự tạo nên máy chưa cài CA sẽ thấy cảnh báo.

CHƯƠNG 11. TRIỂN KHAI HỆ THỐNG

11.1. Sơ đồ triển khai

Hệ thống chạy trên một laptop Windows 10 có Docker Desktop, gồm 4 container (Hình 11.1). Chỉ hai cổng được mở ra mạng LAN: 8883 (MQTT/TLS cho thiết bị) và 8443 (HTTPS cho trình duyệt; 8088 chỉ chuyển hướng sang HTTPS). Cổng REST 8080, MQTT 1883 và PostgreSQL chỉ gắn vào 127.0.0.1 để phục vụ kiểm thử trên chính máy chủ.



Hình 11.1. *Sơ đồ triển khai*

11.2. Các bước triển khai

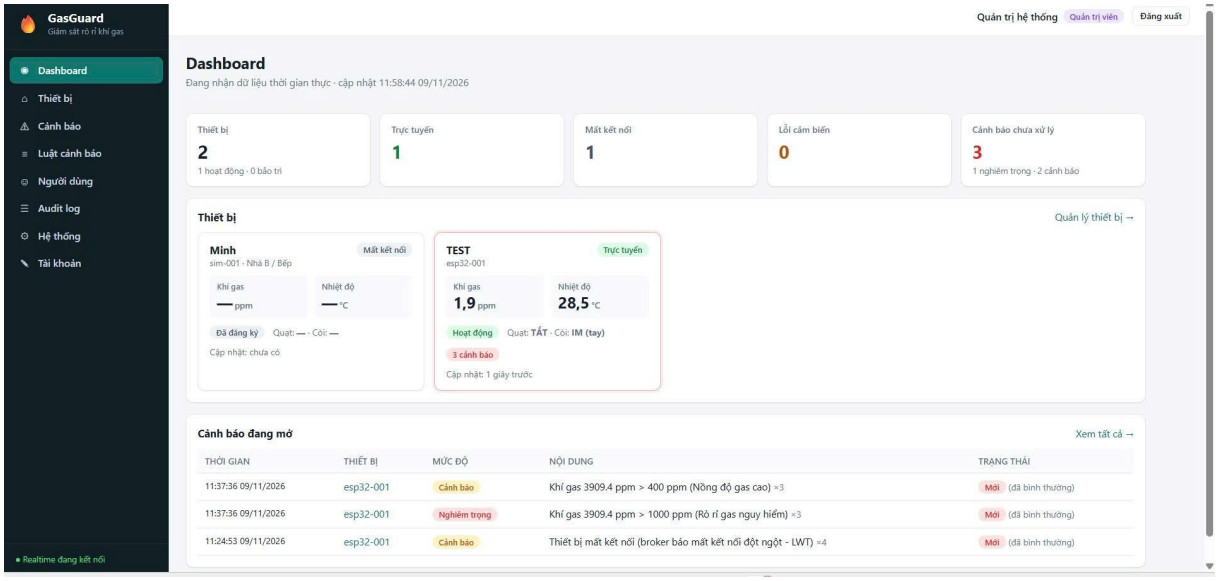
1. Sao chép .env.example thành .env, đặt mật khẩu CSDL, broker, JWT_SECRET (≥ 32 ký tự ngẫu nhiên), mật khẩu admin, địa chỉ broker cho thiết bị và (tùy chọn) tài khoản Gmail.
2. Tạo chứng chỉ TLS bằng deployment/certs/gen-certs.sh (chạy trong container Alpine, ghi mọi IP LAN sẽ dùng); script sinh CA, chứng chỉ máy chủ và firmware/gas-node/ca_cert.h.
3. Khởi tạo Dynamic Security lần đầu: tạo tài khoản admin của broker và tài khoản backend với ACL tối thiểu.
4. Chạy docker compose --env-file ../.env --profile app up -d --build; kiểm tra backend ở trạng thái healthy.
5. Đăng nhập https://<IP-LAN>:8443, đăng ký thiết bị, cấp xác thực, điền secrets.h và nạp firmware bằng Arduino IDE (board ESP32 Dev Module).

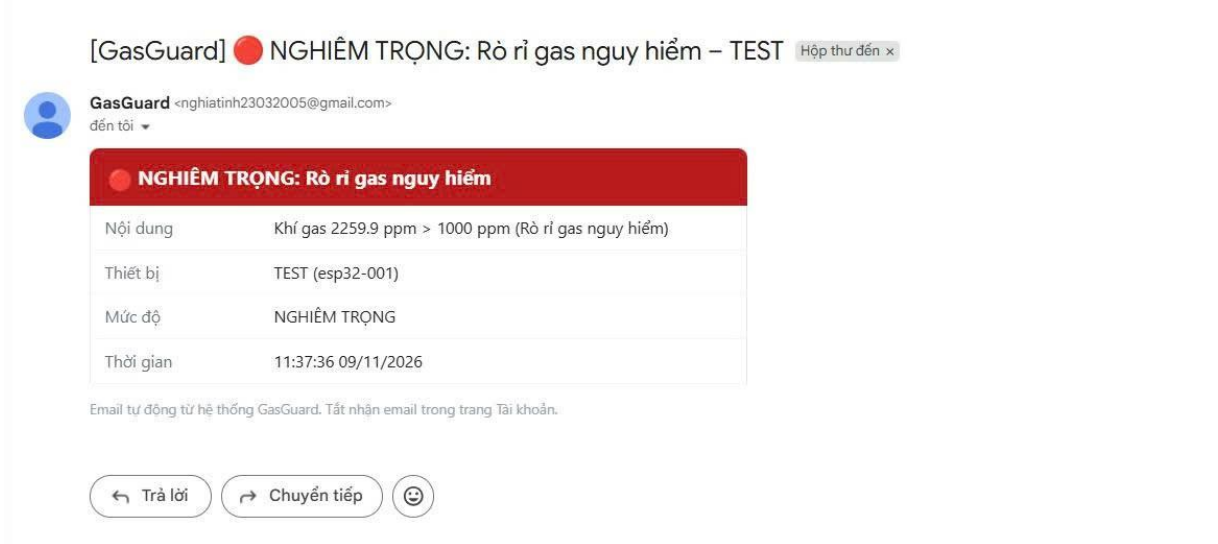
11.3. Giao diện người dùng

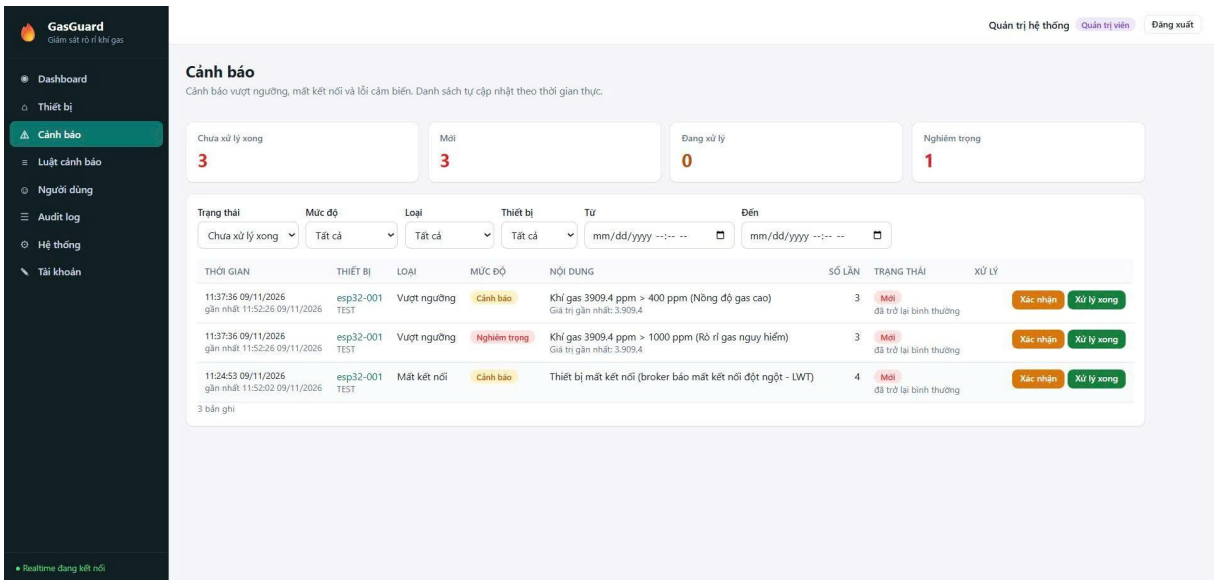
Bảng 11.1. *Các màn hình của giao diện web*

Màn hình	Đường dẫn	Vai trò

Màn hình	Đường dẫn	Vai trò
Đăng nhập / Đăng ký	/login, /register	Tất cả
Dashboard thời gian thực	/	Tất cả
Danh sách, đăng ký thiết bị	/devices	Tất cả (đăng ký: ADMIN, OWNER)
Chi tiết thiết bị: tổng quan, dữ liệu & biểu đồ, điều khiển, cấu hình, cảnh báo, xác thực MQTT, chia sẻ, lịch sử	/devices/:code	Theo quyền trên thiết bị
Cảnh báo; Luật cảnh báo	/alerts, /alert-rules	Tất cả (xử lý theo quyền)
Người dùng; Audit log; Hệ thống	/users, /audit, /system	ADMIN
Tài khoản	/account	Tất cả







Hình 11.2. *Giao diện web: dashboard thời gian thực, chi tiết thiết bị (biểu đồ, điều khiển) và cảnh báo*

11.4. Quản lý mã nguồn

Repository: https://github.com/nghianguyen2303/iot-gassguard-monitor, gồm đủ các thư mục firmware/, backend/, frontend/, database/, deployment/, hardware/, tests/, docs/ và .env.example theo mục 8.1 đề bài. Phiên bản cuối được gắn tag **v1.0.0**.

CHƯƠNG 12. KIỂM THỬ VÀ ĐÁNH GIÁ

12.1. Kế hoạch và môi trường kiểm thử

Kiểm thử gồm bốn bộ: chức năng (F1–F9) và bảo mật (S1–S9) tự động bằng tests/e2e/run_e2e.py; phi chức năng (P1–P8) bằng tests/perf/run_perf.py; thủ công với ESP32 thật (H1–H10). Các bộ tự động dùng thiết bị mô phỏng chạy **đúng giao thức của firmware** (tests/simulator/simulator.py) và client WebSocket đăng nhập giống trình duyệt. Kết quả lưu dạng .md và .json trong tests/reports/ [26].

Môi trường: laptop Windows 10, Python 3.11.7; hệ thống chạy bằng Docker Compose (Mosquitto 2.0.20, PostgreSQL 16, backend Spring Boot 3.5 với heap 384 MB, nginx). Lần chạy được báo cáo: 08/11/2026 (chức năng, bảo mật lúc 13:12; phi chức năng lúc 13:18).

12.2. Kiểm thử chức năng

Bảng 12.1. *Kết quả kiểm thử chức năng (18/18 ca đạt cùng bảng 12.2)*

Mã	Ca kiểm thử	Yêu cầu	Số bước	Kết quả
F1	Đăng nhập và phân quyền theo vai trò	7.1, 4.4a, 8.4-1	8	Đạt
F2	Đăng ký, cấp xác thực và kích hoạt thiết bị	7.1, 4.4b, 8.4-2	8	Đạt
F3	Thiết bị gửi telemetry; backend kiểm tra, lưu, truy vấn	7.1, 4.4c, 8.4-3	8	Đạt
F4	Dashboard cập nhật thời gian thực (WebSocket)	7.1, 4.4d, 8.4-4	4	Đạt

Mã	Ca kiểm thử	Yêu cầu	Số bước	Kết quả
F5	Gửi lệnh điều khiển và nhận ACK	7.1, 4.4e, 8.4-5	8	Đạt
F6	Thay đổi cấu hình thiết bị từ xa	4.4b, 8.4-6	4	Đạt
F7	Phát hiện offline và trở lại online	7.1, 4.4d, 8.4-7	5	Đạt
F8	Sinh cảnh báo theo luật và xử lý cảnh báo	7.1, 4.4d, 8.4-8	7	Đạt
F9	Tạm khóa và ngừng sử dụng thiết bị	7.1, 4.4b, 4.5, 8.4-10	12	Đạt

12.3. Kiểm thử bảo mật

Đề bài yêu cầu tối thiểu hai kiểm thử bảo mật; nhóm thực hiện 9 ca, bao gồm đủ năm ví dụ trong mục 4.5 [1].

Bảng 12.2. *Kết quả kiểm thử bảo mật*

Mã	Ca kiểm thử	Bằng chứng tiêu biểu	Kết quả
S1	Không truy cập được thiết bị của người khác	GET /api/devices/{thiết bị người khác} → HTTP 403	Đạt
S2	Người được chia sẻ chỉ xem, không điều khiển	Chia sẻ → 201; người được chia sẻ đọc được dữ liệu nhưng gửi lệnh, đổi cấu hình, đổi mật khẩu thiết bị, chuyển vòng đời → 403	Đạt
S3	Từ chối token thiếu, sai, giả mạo, hết hạn, đã thu hồi	Không có token, token rác, sửa chữ ký, tự sửa vai trò thành ADMIN, hết hạn, đã đăng xuất → 401; WebSocket từ chối CONNECT với token sai	Đạt
S4	Broker từ chối kết nối sai, publish/subscribe sai topic	Thiết bị A publish vào topic thiết bị B → PUBACK 135 (Not authorized)	Đạt
S5	Từ chối dữ liệu không hợp lệ	9 loại bản tin lỗi (JSON hỏng, thiếu/sai ts, ngoài dải, sai đơn vị, trường lạ, sai deviceId, quá 4 KB) bị từ chối, không lưu CSDL; REST dữ liệu sai → 400	Đạt
S6	Khóa tạm khi dò mật khẩu	Sai 5 lần → [401, 401, 401, 401, 423], khóa 15 phút	Đạt
S7	Mật khẩu cũ của thiết bị hết hiệu lực	Đổi mật khẩu thiết bị → phiên dùng mật khẩu cũ bị ngắt, kết nối lại bằng mật khẩu cũ bị từ chối, mật khẩu mới hoạt động	Đạt
S8	Audit log và mật khẩu được băm	Audit có DEVICE_REGISTER; password_hash dạng BCrypt $2a$12$…	Đạt
S9	Kết nối mã hóa TLS	MQTT/TLS kiểm tra chứng chỉ bằng CA nhóm → CONNACK 0, TLSv1.3, TLS_AES_256_GCM_SHA384; kết nối không mã hóa và chứng chỉ giả bị từ chối	Đạt

12.4. Kiểm thử phi chức năng

Bảng 12.3. *Tổng hợp chỉ tiêu phi chức năng*

Mã	Chỉ tiêu	Kết quả đo	So với mục tiêu
P1	Độ trễ cảm biến → dashboard	P95 24,5 ms; nhận 200/200	Đạt (≤ 1 000 ms)
P2	Độ trễ lệnh điều khiển	P95 khứ hồi 151,9 ms; 50/50 thành công	Đạt (≤ 1 000 ms)
P3	Tỷ lệ mất bản tin	QoS1: 0,00 % (1 000 bản tin, 50/s)	Đạt (0 %)
P4	Mất và khôi phục mạng	OFFLINE sau 1,0 s; gửi bù 20/20	Đạt (≤ 30 s)

Mã	Chỉ tiêu	Kết quả đo	So với mục tiêu
P5	Bản tin trùng lặp	Lưu 200/200, bỏ qua 400 số đo trùng	Đạt
P6	CPU, bộ nhớ	RAM backend tối đa 375 MiB; broker < 4 MiB	Đạt (≤ 512 MB)
P7	Chạy liên tục 10 phút	Mất 0,000 %; RAM +3,4 %	Đạt
P8	Số thiết bị đồng thời	10, 50, 100 thiết bị: mất 0 %; 100 thiết bị: P95 758 ms	Đạt

Bảng 12.4 tóm tắt phương pháp đo, dữ liệu và nhận xét cho từng chỉ tiêu; dữ liệu đo chi tiết (min, trung bình, P50, P95, max và từng mức tải) nằm ở Phụ lục B và trong tests/reports/perf-20261108-131845.json [26].

Bảng 12.4. *Phương pháp đo, dữ liệu và nhận xét cho các chỉ tiêu phi chức năng*

Mã	Phương pháp đo	Dữ liệu đo	Nhận xét
P1	1 thiết bị mô phỏng gửi 200 bản tin QoS1 mỗi 200 ms; client STOMP ghi thời điểm nhận; độ trễ = nhận − ts	Thiết bị → backend P95 7,4 ms; đầu-cuối P50 14,4 ms, P95 24,5 ms; nhận 200/200	Đạt (≤ 1 s). Chặng WebSocket chỉ thêm ~9,5 ms; ESP32 qua WiFi cộng thêm vài chục ms
P2	50 lệnh FAN_SET/PING qua REST, waitMs=5000; backend ghi latencyMs, client đo khứ hồi	ACK P50 53,5 ms, P95 65,1 ms; khứ hồi P95 151,9 ms; 50/50 SUCCESS	Đạt. Một mẫu ngoại lai 2,2 s (lệnh thứ 12), 49 lệnh còn lại khứ hồi ≤ 155 ms
P3	Mỗi mức QoS gửi 1 000 bản tin ở 50/s (gấp 250 lần thực tế); đếm bản ghi trong CSDL	QoS1: 1 000/1 000; QoS0: 1 000/1 000; ước lượng theo seq khớp	Đạt (0 %). QoS0 chỉ dùng cho heartbeat
P4	Ngắt TCP đột ngột; mất mạng 20 s có đệm; khởi động lại broker, backend	OFFLINE sau 1,0 s; gửi bù 20/20; broker: nối lại 4,0 s, lưu 24/24; backend: 16,2 s, lưu 50/50	Đạt (≤ 30 s). ESP32 rút điện: LWT sau ≤ 1,5 × keep-alive 15 s, watchdog 30 s dự phòng
P5	Gửi 200 bản tin rồi gửi lại y hệt (cùng seq, ts)	Lưu 200/200; 400 số đo trùng bị bỏ qua	Đạt nhờ UNIQUE + ON CONFLICT DO NOTHING; không sinh cảnh báo lặp
P6	docker stats mỗi ~2 s lúc nghỉ và ở 10/50/100 thiết bị	RAM backend 370–375 MiB; broker ≤ 3,5 MiB; CPU backend TB 7–67 % một nhân	Đạt. Heap giới hạn 384 MB; firmware dùng ~15 % RAM, 74 % flash
P7	5 thiết bị × 1 bản tin/s trong 10 phút	Gửi 3 000, lưu 3 000, 0 bị từ chối; RAM 358 → 370 MiB	Đạt. P95 phút đầu 3 964 ms do JVM nguội, sau đó 94–184 ms
P8	10/50/100 thiết bị × 1 bản tin/s trong 60 s	10 thiết bị: 600/600; 50 thiết bị: 3 000/3 000; 100 thiết bị: 6 000/6 000 (mất 0 %), P95 758 ms	Đạt ở 100 thiết bị (≈ 500 thiết bị thật chu kỳ 5 s). Giới hạn tiếp theo là ghi CSDL từng giao dịch, subscriber tuần tự; khi cần nhiều thiết bị hơn: ghi theo lô, hàng đợi, shared subscription

12.5. Kiểm thử với thiết bị thật (ESP32)

Các ca H1–H10 được làm thủ công với node esp32-001, firmware 1.0.0, qua WiFi 2,4 GHz, theo kịch bản trong docs/test-plan.md. Ngày kiểm thử: 08/11/2026; người thực hiện: Nguyễn Đại Nghĩa; IP máy chủ: 192. 168. 5. 176.

Bảng 12.5. *Kết quả kiểm thử với thiết bị thật*

Mã	Ca kiểm thử	Kết quả mong đợi	Số liệu quan sát	Kết quả
H1	Kích hoạt thiết bị thật	Serial Đã kết nối broker; web ACTIVE, ONLINE, fw 1.0.0	Từ cấp nguồn đến kết nối broker: **11,3 s** (WiFi 8,8 s → NTP 10,9 s → broker 11,3 s); MQTT/TLS tới 192.168.1.245:8884; web: ACTIVE, ONLINE, fw 1.0.0	Đạt
H2	Telemetry + realtime	Số đo cập nhật mỗi chu kỳ không cần tải lại; độ trễ TB < 1 s	Độ trễ TB: ≈ 57 ms (1.432 bản ghi realtime; giá trị thô −238 ms do đồng hồ máy chủ chậm 295 ms so với NTP, đã hiệu chỉnh)	Đạt
H3	Điều khiển quạt, còi	Relay đóng, quạt quay/dừng, còi kêu/im; web “Thành công”	Quạt: 120 / 52 / 67 ms (BẬT / TẮT / Tự động); Còi: 56 / 81 / 126 / 96 / 132 ms; trung bình 91 ms	Đạt
H4	Cấu hình từ xa	Đổi chu kỳ 3 s, ngưỡng 300 ppm → “Đã áp dụng”	Cấu hình v2 (chu kỳ 3 s, ngưỡng 390 ppm) gửi 12:16:35.708 → "Đã áp dụng" 12:16:35.779 (71 ms); chu kỳ gửi đổi 5,0 s → 3,0 s từ seq 486	Đạt
H5	Cảnh báo gas	LED đỏ, còi, quạt tự bật; cảnh báo web; email	Gas đỉnh 10000 ppm (trần thang đo, 11:39:07; cảnh báo CRITICAL ghi 3909,4 ppm > 1000 ppm); email sau [không còn log]	Đạt
H6	Offline	Rút USB → Ngoại tuyến + cảnh báo; cắm lại tự online	OFFLINE sau 27 s (bản tin cuối 12:19:14 → OFFLINE/LWT 12:19:41); cắm lại ONLINE 12:20:51, cảnh báo tự xóa	Đạt
H7	Mất mạng, gửi bù	Dừng broker 40 s; báo động tại chỗ vẫn chạy; Serial Gửi bù …	Gửi bù 8 bản (seq 176–183, đo 11:49:04–11:50:31, nhận dồn lúc 11:50:33); không trùng seq	Đạt
H8	Lỗi cảm biến	Rút dây DATA DS18B20 → LED đỏ nháy, web “Lỗi”, cảnh báo FAULT	LỖI cảm biến temp: DS18B20 không phản hồi (đọc được -127)	Đạt
H9	Khóa / thu hồi	Tạm khóa → Serial Broker TỪ CHỐI (mã 5); mở khóa tự kết nối lại	Broker TỪ CHỐI (mã 5): sai mật khẩu, hoặc thiết bị bị tạm ngưng / thu hồi / đổi mật khẩu	Đạt
H10	Quá nhiệt	Đầu DS18B20 trong nước ~60 °C: > 45 °C báo động tại chỗ; > 50 °C cảnh báo web	Nhiệt độ đỉnh 55 °C	Đạt

Hình 12.1. *Bằng chứng kiểm thử với thiết bị thật (Serial Monitor, web, email)*

12.6. Đánh giá chung

Đối chiếu với tiêu chí ở mục 2.6: các quy trình bắt buộc của mục 7.1 chạy đầu-cuối; toàn bộ NFR1–NFR8 đạt; NFR9 đạt ở mức 50 thiết bị; 9/9 ca bảo mật đạt, kiểm tra ở cả ba lớp broker (ACL, tài khoản riêng), API (JWT, phân quyền, kiểm tra đầu vào) và lưu trữ (BCrypt, audit log). Điểm cần cải thiện là thông lượng ghi CSDL khi số thiết bị lớn và việc số liệu tải được đo trên một laptop chứ không phải máy chủ riêng.

CHƯƠNG 13. CHỨC NĂNG NÂNG CAO

Nhóm không triển khai OTA và AI (mục 5.1, 5.2 đề bài) để tập trung hoàn thiện các yêu cầu bắt buộc. Các hướng nâng cao trong mục 5.3 đã triển khai:

Bảng 13.1. *Chức năng nâng cao đã triển khai*

Hướng nâng cao	Nội dung	Bằng chứng
Edge computing	Luật báo động có trễ, tự bật quạt, lọc nhiễu ADC, hiệu chuẩn R0 chạy trên ESP32, độc lập với mạng	Mục 5.6, H5, H7
Device shadow	actuator_state (trạng thái quạt/còi, chế độ) và cấu hình có phiên bản được đồng bộ từ heartbeat, ACK; topic config retained	F5, F6
Rule engine cảnh báo	Luật chung và theo thiết bị, 4 toán tử, 3 mức độ, gộp cảnh báo lặp, cảnh báo OFFLINE/FAULT, email	F8
Xử lý thời gian thực	Kiến trúc hướng sự kiện, WebSocket STOMP có kiểm tra quyền theo kênh	F4, P1
Mất mạng và đồng bộ lại	Bộ đệm 120 bản trên thiết bị, phiên MQTT bền ở backend, chống trùng ở CSDL	P4, P5
Triển khai bằng Docker	4 container, healthcheck, giới hạn heap, bí mật qua .env	Chương 11
Mở rộng bằng thiết bị mô phỏng	Simulator đúng giao thức, tới 100 thiết bị đồng thời	P8
Bảo mật nâng cao	TLS với CA riêng, tài khoản MQTT động bằng Dynamic Security	S4, S7, S9

CHƯƠNG 14. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

Bảng 14.1. *Hạn chế hiện tại và hướng phát triển*

Hạn chế	Hướng phát triển
MQ-2 chỉ cho nồng độ ước lượng, chịu ảnh hưởng nhiệt độ, độ ẩm; không chọn lọc riêng LPG	Bù nhiệt độ, độ ẩm; dùng MQ-6 hoặc cảm biến xúc tác đạt chuẩn; hiệu chuẩn bằng khí chuẩn
Thông lượng giảm ở 100 bản tin/s (mất 8,22 %)	Ghi CSDL theo lô, hàng đợi nội bộ, shared subscription; dùng TimescaleDB khi dữ liệu lớn
CA tự tạo, thiết bị xác thực bằng mật khẩu	Chứng chỉ client cho từng thiết bị (mTLS); triển khai máy chủ có tên miền và chứng chỉ công cộng

Hạn chế	Hướng phát triển
JWT trong localStorage	Cookie HttpOnly, SameSite, refresh token
Chưa có OTA, AI	OTA có ký số và rollback; phát hiện bất thường (rò rỉ nhỏ kéo dài) từ chuỗi thời gian
Một node, nguồn USB, chưa có vỏ bảo vệ	Thêm node, mạch in (PCB), nguồn riêng có tụ lọc và diode dập, vỏ đặt sát sàn
Chưa có thông báo đẩy trên điện thoại	Ứng dụng di động hoặc Zalo OA; Telegram khi mạng ổn định

CHƯƠNG 15. KẾT LUẬN

Đồ án đã xây dựng thành công GasGuard, một nguyên mẫu IoT đầu-cuối cho bài toán giám sát và cảnh báo rò rỉ khí gas. Hệ thống gồm node ESP32 thật với hai cảm biến (MQ-2, DS18B20) và ba cơ cấu đầu ra (quạt qua relay, còi, LED), giao tiếp MQTT qua TLS với tài khoản riêng từng thiết bị, backend tự xây dựng quản lý đầy đủ người dùng ba vai trò, vòng đời thiết bị sáu trạng thái, dữ liệu, cảnh báo và lệnh có xác nhận, cùng giao diện web thời gian thực.

Về mặt kỹ thuật, nhóm rút ra ba bài học chính. Một là phản ứng an toàn phải đặt ở biên: luật báo động trên thiết bị giúp hệ thống vẫn bảo vệ được người dùng khi mạng hoặc máy chủ gặp sự cố. Hai là độ tin cậy cần được thiết kế ở mọi tầng: QoS1, phiên bền, bộ đệm trên thiết bị và ràng buộc chống trùng trong CSDL cùng nhau bảo đảm không mất và không trùng dữ liệu. Ba là bảo mật phải kiểm chứng được: mỗi biện pháp trong mô hình đe dọa đều có ca kiểm thử tự động tương ứng. Kết quả đo cho thấy hệ thống đạt các chỉ tiêu đặt ra ở quy mô hộ gia đình đến khu nhà trọ, đồng thời chỉ ra rõ giới hạn thông lượng và hướng cải tiến khi mở rộng.

TÀI LIỆU THAM KHẢO

[1] Bộ môn giảng dạy học phần IoT và ứng dụng, “Đề bài đồ án kết thúc học phần – Học phần IoT và ứng dụng,” tài liệu học phần, 2026.

[2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, “Internet of Things: A survey on enabling technologies, protocols, and applications,” *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015, doi: 10.1109/COMST.2015.2444095.

[3] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.

[4] Espressif Systems, *ESP32 Series Datasheet*. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf (truy cập: 08/11/2026).

[5] Espressif Systems, “Analog to Digital Converter (ADC),” *ESP-IDF Programming Guide (ESP32)*. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/adc_oneshot.html (truy cập: 08/11/2026).

[6] Hanwei Electronics Co., Ltd., *Technical Data MQ-2 Gas Sensor*, datasheet. Zhengzhou, China.

[7] Maxim Integrated (Analog Devices), *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, datasheet. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf (truy cập: 08/11/2026).

[8] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in *Proc. 2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria, 2017, pp. 1–7, doi: 10.1109/SysEng.2017.8088251.

[9] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)," IETF RFC 7252, Jun. 2014, doi: 10.17487/RFC7252.

[10] A. Banks and R. Gupta, Eds., *MQTT Version 3.1.1*, OASIS Standard, 29 Oct. 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[11] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011, doi: 10.17487/RFC6455.

[12] STOMP Project, *STOMP Protocol Specification, Version 1.2*. [Online]. Available: https://stomp.github.io/stomp-specification-1.2.html (truy cập: 08/11/2026).

[13] R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," *Journal of Open Source Software*, vol. 2, no. 13, p. 265, 2017, doi: 10.21105/joss.00265.

[14] Eclipse Foundation, "Dynamic Security plugin," *Eclipse Mosquitto Documentation*. [Online]. Available: https://mosquitto.org/documentation/dynamic-security/ (truy cập: 08/11/2026).

[15] VMware Tanzu (Broadcom), *Spring Boot Reference Documentation*. [Online]. Available: https://docs.spring.io/spring-boot/ (truy cập: 08/11/2026).

[16] VMware Tanzu (Broadcom), *Spring Security Reference Documentation*. [Online]. Available: https://docs.spring.io/spring-security/reference/ (truy cập: 08/11/2026).

[17] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, "INSERT – ON CONFLICT Clause." [Online]. Available: https://www.postgresql.org/docs/16/sql-insert.html (truy cập: 08/11/2026).

[18] Docker Inc., *Docker Compose Documentation*. [Online]. Available: https://docs.docker.com/compose/ (truy cập: 08/11/2026).

[19] Maxim Integrated, "Guidelines for reliable long line 1-Wire networks," Tutorial 148. [Online]. Available: https://www.analog.com/en/resources/technical-articles/guidelines-for-reliable-long-line-1wire-networks.html (truy cập: 08/11/2026).

[20] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.

[21] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, National Institute of Standards and Technology, Gaithersburg, MD, USA, Jun. 2017, doi: 10.6028/NIST.SP.800-63b.

[22] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015, doi: 10.17487/RFC7519.

[23] OWASP Foundation, "OWASP API Security Top 10 – 2023." [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (truy cập: 08/11/2026).

[24] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.

[25] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 8446, Aug. 2018, doi: 10.17487/RFC8446.

[26] Nhóm GasGuard, "iot-gasguard: mã nguồn, kế hoạch và báo cáo kiểm thử (docs/test-plan.md, docs/test-report.md, tests/reports/)," GitHub repository, phiên bản v1.0.0, 2026. [Online]. Available: https://github.com/nghianguyen2303/iot-gassguard-monitor

PHỤ LỤC A. DANH SÁCH ĐẦY ĐỦ API

Bảng A.1. *53 endpoint REST của backend*

Phương thức	Đường dẫn	Mô tả / quyền
POST	/api/auth/register	Đăng ký (công khai, vai trò VIEWER)
POST	/api/auth/login	Đăng nhập, trả JWT (công khai)
POST	/api/auth/logout	Đăng xuất, thu hồi token
GET / PUT	/api/auth/me	Xem / sửa hồ sơ
PUT	/api/auth/password	Đổi mật khẩu
GET / DELETE	/api/auth/me/email-alerts	Trạng thái / hủy nhận email
POST	/api/auth/me/email-alerts/request, /confirm, /test	Yêu cầu, xác nhận mã, gửi thử
GET / DELETE	/api/auth/me/telegram	Trạng thái / hủy liên kết Telegram
POST	/api/auth/me/telegram/link, /test	Liên kết, gửi thử
GET / POST	/api/admin/users	Danh sách / tạo người dùng (ADMIN)
PUT	/api/admin/users/{id}/role	Đổi vai trò (ADMIN)
POST	/api/admin/users/{id}/lock, /unlock	Khóa / mở khóa (ADMIN)
GET / POST	/api/devices	Danh sách theo quyền / đăng ký (ADMIN, OWNER)
GET / PUT	/api/devices/{code}	Chi tiết (xem) / cập nhật (quản lý)
POST	/api/devices/{code}/provision	Cấp thông tin xác thực (quản lý)
POST	/api/devices/{code}/credentials/rotate	Đổi mật khẩu thiết bị (quản lý)
GET	/api/devices/{code}/credentials	Lịch sử xác thực (quản lý)
POST	/api/devices/{code}/lifecycle	Chuyển trạng thái vòng đời (quản lý)
GET / POST	/api/devices/{code}/shares	Danh sách / thêm chia sẻ (quản lý)
DELETE	/api/devices/{code}/shares/{username}	Thu hồi chia sẻ (quản lý)

Phương thức	Đường dẫn	Mô tả / quyền
GET	/api/devices/{code}/telemetry, /latest, /series, /stats	Dữ liệu thô, mới nhất, chuỗi biểu đồ, thống kê (xem)
GET	/api/devices/{code}/telemetry/export.csv	Xuất CSV (xem, ghi audit)
GET / POST	/api/devices/{code}/commands	Lịch sử (xem) / gửi lệnh (quản lý)
GET	/api/devices/{code}/commands/{id}	Chi tiết lệnh (xem)
GET / PUT	/api/devices/{code}/config	Xem (xem) / đổi cấu hình (quản lý)
GET	/api/devices/{code}/activity	Lịch sử hoạt động thiết bị (xem)
GET	/api/alerts, /api/alerts/summary	Tra cứu, tổng hợp cảnh báo (theo quyền xem)
POST	/api/alerts/{id}/ack, /resolve	Xác nhận, xử lý (quản lý)
GET / POST	/api/alert-rules	Danh sách / tạo luật
PUT / DELETE	/api/alert-rules/{id}	Sửa / xóa luật (chung: ADMIN; riêng: quản lý)
GET	/api/dashboard	Tổng quan theo quyền
GET	/api/admin/audit-logs	Audit log toàn hệ thống (ADMIN)
GET	/api/system/ingest-stats	Thống kê tiếp nhận MQTT (ADMIN)

Ghi chú: một dòng có thể gộp nhiều endpoint cùng tài nguyên; tổng số endpoint là 53. Kênh WebSocket: /ws (STOMP), đăng ký /topic/devices/{code} cần quyền xem thiết bị.

PHỤ LỤC B. DỮ LIỆU ĐO CHI TIẾT CÁC CHỈ TIÊU PHI CHỨC NĂNG

P1 – Độ trễ từ cảm biến đến dashboard

Phương pháp: một thiết bị mô phỏng gửi 200 bản tin QoS1, mỗi 200 ms, ghi ts; một client STOMP đăng ký kênh của thiết bị và ghi thời điểm nhận. Độ trễ = thời điểm dashboard nhận − ts; backend ghi thêm received_at để tách phần MQTT.

Bảng B.1. *Dữ liệu đo P1 (ms, 200 mẫu)*

Chặng	Min	TB	P50	P95	Max
Thiết bị → backend nhận	1,0	4,8	4,9	7,4	11,2
Thiết bị → dashboard (đầu-cuối)	6,1	15,9	14,4	24,5	196,3

Nhận xét: phần đẩy WebSocket chỉ thêm khoảng 9,5 ms. Với ESP32 thật qua WiFi, độ trễ tăng thêm thời gian truyền WiFi (thường vài chục ms), vẫn xa dưới mục tiêu 1 s.

P2 – Độ trễ lệnh điều khiển

Phương pháp: gửi 50 lệnh (FAN_SET bật/tắt xen kẽ PING) qua REST với waitMs=5000; backend ghi latencyMs từ lúc gửi đến lúc nhận ACK, client đo thêm thời gian khứ hồi của lời gọi API.

Bảng B.2. *Dữ liệu đo P2 (ms, 50 mẫu)*

Đại lượng	Min	TB	P50	P95	Max
Backend gửi → nhận ACK	7	94,8	53,5	65,1	2 169
Khứ hồi API (người dùng)	120,2	182,1	139,7	151,9	2 301,0

Nhận xét: 50/50 lệnh thành công. Giá trị lớn nhất (2,2 s) là một mẫu ngoại lai duy nhất ở lệnh thứ 12, nhiều khả năng do một lần tạm dừng của JVM hoặc mạng cục bộ; 49 lệnh còn lại có thời gian ACK ≤ 72 ms và khứ hồi ≤ 155 ms; chênh lệch giữa hai dòng là thời gian lưu CSDL, kiểm tra quyền và ghi audit.

P3, P5 – Mất bản tin và bản tin trùng

Phương pháp P3: với mỗi mức QoS, gửi 1 000 bản tin liên tục ở 50 bản tin/s (nhanh gấp 250 lần cấu hình thực tế 1 bản tin/5 s), đếm số bản ghi trong CSDL. **Kết quả:** QoS1 và QoS0 đều lưu 1 000/1 000 (0,00 %); ước lượng mất theo seq của chính hệ thống khớp số đếm thực tế. **P5:** gửi 200 bản tin rồi gửi lại y hệt (cùng seq, ts); CSDL vẫn chỉ có 200 bản ghi, 400 số đo trùng (2 chỉ số × 200) bị bỏ qua và không sinh cảnh báo lặp.

P4 – Mất và khôi phục mạng

Bảng B.3. *Dữ liệu đo P4*

Tình huống	Đại lượng	Giá trị	Cơ chế
Ngắt TCP đột ngột (như mất điện/WiFi)	Chuyển OFFLINE sau	1,0 s	LWT
Thiết bị kết nối lại	Chuyển ONLINE sau	0,1 s	status ONLINE
Dữ liệu đo trong 20 s mất mạng	Đã lưu sau khi gửi bù	20/20	Đệm + giữ ts gốc
Khởi động lại broker	Kết nối lại sau	4,0 s	Tự kết nối lại
Bản tin QoS1 trong lúc broker gián đoạn	Đã lưu	24/24	Hàng đợi client
Khởi động lại backend	Hoạt động lại sau	16,2 s	Phiên MQTT bền
Bản tin gửi trong lúc backend ngừng	Đã lưu	50/50	Broker xếp hàng QoS1

Nhận xét: thiết bị mô phỏng trong P4 đóng kết nối TCP ngay nên broker phát LWT gần như tức thì. Với ESP32 thật bị rút điện, broker chỉ phát hiện khi hết keep-alive (tối đa khoảng 1,5 × 15 s); nếu LWT không đến, watchdog backend vẫn đánh OFFLINE sau 30 s. Cả hai trường hợp đều đạt mục tiêu ≤ 30 s.

P6, P7 – Tài nguyên và độ ổn định

Bảng B.4. *Mức sử dụng tài nguyên theo số thiết bị (P6, lấy mẫu docker stats mỗi ~2 s)*

Số thiết bị	CPU backend TB %	CPU backend max %	RAM backend max (MiB)	CPU broker TB %	RAM broker max (MiB)	CPU CSDL TB %
0 (nghỉ)	7,4	10,9	370,0	0,2	1,7	0,3
10	37,6	246,7	371,5	0,9	1,8	12,0
50	49,4	182,8	373,2	2,8	2,5	22,5
100	67,4	160,2	374,6	3,7	3,5	37,2

CPU tính theo phần trăm một nhân. Backend Java chiếm nhiều RAM nhất nhưng bị giới hạn heap 384 MB nên vừa với máy 8 GB; firmware ESP32 dùng khoảng 15 % RAM và 74 % flash. **P7:** 5 thiết bị, mỗi thiết bị 1 bản tin/giây trong 10 phút: gửi 3 000, lưu 3 000 (mất 0,000 %), không có bản tin bị từ chối, RAM backend tăng từ 358 lên 370 MiB (+3,4 %) và ổn định nhờ giới hạn heap. P95 độ trễ phút đầu cao (3 964 ms) do JVM khởi động nguội, từ phút thứ ba ổn định ở 94–184 ms.

P8 – Số lượng thiết bị đồng thời

Bảng B.5. *Dữ liệu đo P8 (mỗi thiết bị 1 bản tin/s, 60 s, QoS1)*

Thiết bị	Gửi/s	Xử lý/s	Đã gửi	Đã lưu	Mất	P95 (ms)
10	10	10	600	600	0,00 %	187
50	50	50	3 000	3 000	0,00 %	255
100	100	92	6 000	6000	0,00 %	758

Nhận xét: hệ thống xử lý trọn vẹn 100 bản tin/s, tương đương khoảng 500 thiết bị thật ở chu kỳ 5 s, không mất bản tin, P95 độ trễ dưới 1 giây. Độ trễ tăng dần theo tải (187 → 255 → 758 ms) do mỗi bản tin được ghi CSDL trong một giao dịch riêng và luồng subscriber MQTT xử lý tuần tự. Khi cần quy mô lớn hơn có thể ghi theo lô (batch insert), tách hàng đợi nội bộ giữa nhận và ghi, hoặc dùng shared subscription để chạy nhiều consumer song song. Kết quả này phù hợp phạm vi hộ gia đình đến khu nhà trọ đã đặt ra ở Chương 1.

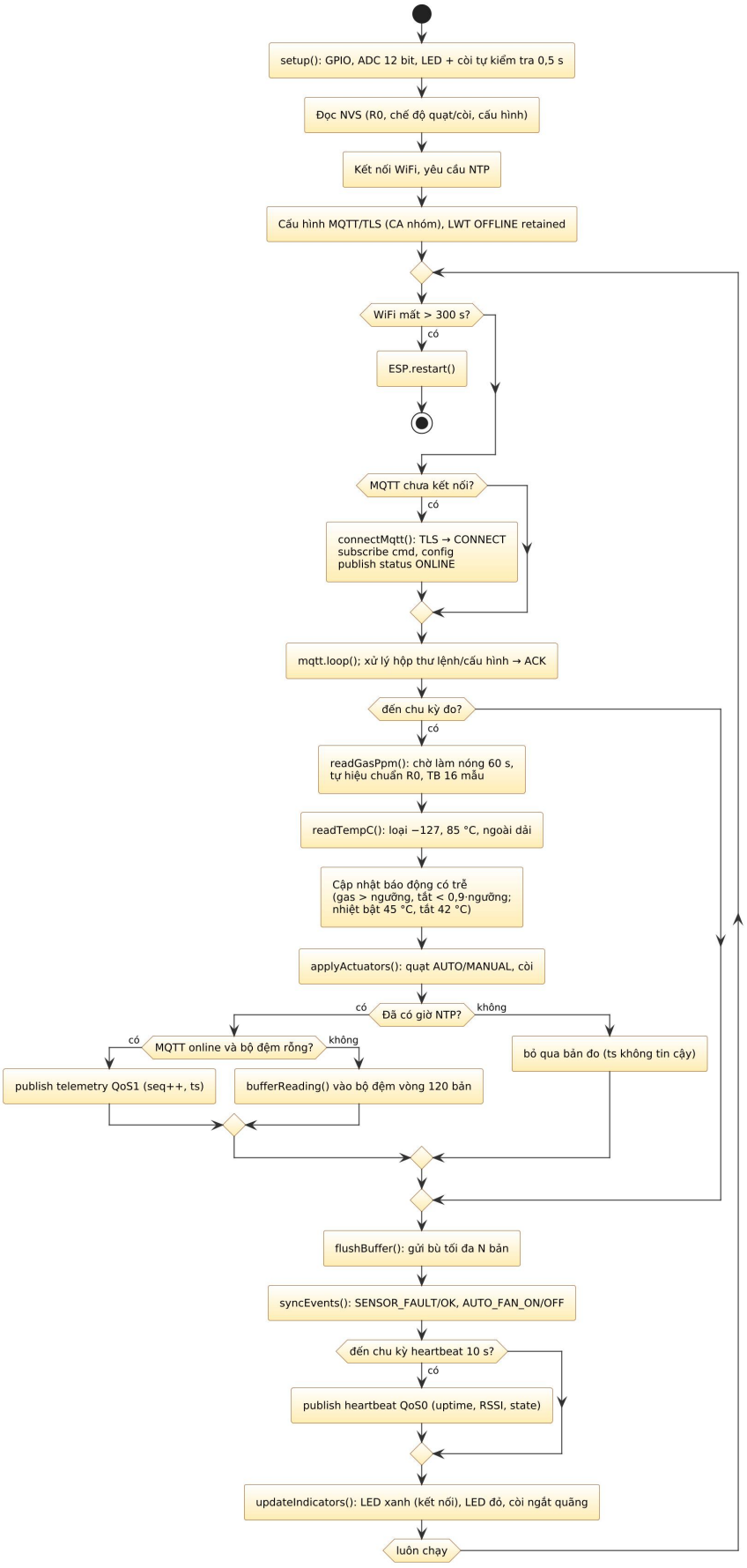
PHỤ LỤC C. LẮP RÁP TRÊN BREADBOARD VÀ SƠ ĐỒ HOẠT ĐỘNG FIRMWARE

Quy ước: breadboard đặt ngang, hàng a ở mép dưới, hàng j ở mép trên, cột 1 bên trái; ray dưới đỏ là 5 V, ray trên đỏ là 3,3 V, **hai ray đỏ không bao giờ nối với nhau**; ESP32 và module relay đặt ngoài breadboard. Bảng C.1 tóm tắt vị trí lắp ráp; bảng đầy đủ 39 dây nằm trong hardware/README.md.

Bảng C.1. *Tóm tắt vị trí lắp ráp trên breadboard*

Khối	Vị trí linh kiện và dây
Nguồn	VIN → ray dưới đỏ; GND → ray dưới xanh; cầu ray đỏ và xanh dưới ở cột 28 ↔ 40; 3V3 → ray trên đỏ (cột 51); ray xanh dưới ↔ ray xanh trên (cột 51).
MQ-2	VCC, GND → ray dưới; AO → e59; 10K b55–b59; 20K a55 → ray xanh dưới; e55 → D34; DO để trống.

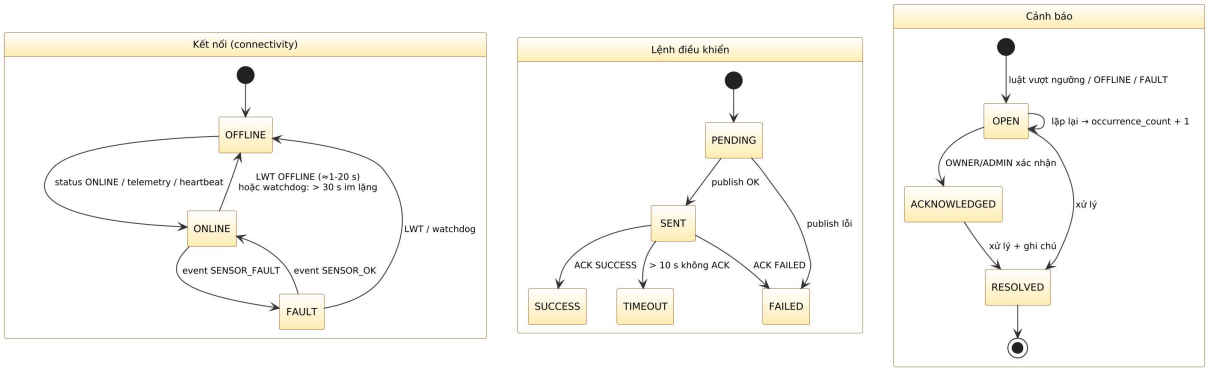
Khối	Vị trí linh kiện và dây
LED	LED đỏ c20 (+) – c21 (−), 220 Ω a21 → GND, e20 → D26; LED xanh c24–c25, 220 Ω a25 → GND, e24 → D27.
Còi	2N2222 E-c30, B-c31, C-c32 (mặt chữ hướng hàng a); 330 Ω d28–d31; còi (−) e32, (+) e35; a30 → GND; a35 → 5 V; e28 → D25.
Relay, quạt	Relay VCC/GND → ray dưới cột 12; IN → e15, a15 → D33; COM → 5 V (cột 14); NO → dây đỏ quạt; dây đen quạt → GND (cột 14); NC để trống.
DS18B20	DATA (vàng) f45, VCC (đỏ) f47, GND (đen) f49; hai điện trở 10K song song g45–g47 và h45–h47; j47 → ray trên 3,3 V; j49 → GND; i45 → D4.



Hình C.1. *Sơ đồ hoạt động của vòng lặp firmware*

PHỤ LỤC D. MÁY TRẠNG THÁI KẾT NỐI, LỆNH VÀ CẢNH BÁO

Các máy trạng thái phụ trợ của hệ thống (kết nối, lệnh điều khiển, cảnh báo) được trình bày trong Hình D.1.



Hình D.1. *Trạng thái kết nối, lệnh điều khiển và cảnh báo*

PHỤ LỤC E. THƯ VIỆN, TÀI NGUYÊN BÊN THỨ BA

Bảng E.1. *Thư viện và tài nguyên bên thứ ba*

Thành phần	Thư viện / phần mềm (giấy phép)
Firmware	Arduino-ESP32 3.x (LGPL-2.1); MQTT by Joel Gaehwiler 2.5 (MIT); ArduinoJson 7 (MIT); OneWire (MIT-style); DallasTemperature (LGPL-2.1)
Backend	Spring Boot 3.5.16, Spring Security, Spring Data JPA, Spring WebSocket, Flyway, PostgreSQL JDBC, jjwt 0.12.6, Eclipse Paho MQTT 1.2.5, Eclipse Angus Mail, Lombok (Apache-2.0 / EPL / MIT / BSD)
Frontend	React 18.3, React Router 6.28, Recharts 2.13, Vite 5.4 (MIT); @stomp/stompjs 7.0 (Apache-2.0)
Hạ tầng	Eclipse Mosquitto 2.0.20 (EPL/EDL), PostgreSQL 16, nginx 1.27 (ảnh Docker chính thức)
Kiểm thử	paho-mqtt 2.1 (EPL/EDL), websocket-client 1.x (Apache-2.0)